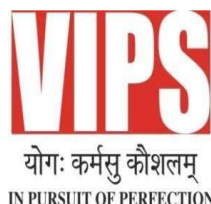


Project Report
On
MediAI – An AI based Medical Diagnosis System

Submitted in partial fulfillment of the
Requirement for the award of the degree of
Masters of Computer Application



Guided By:

Dr. Meenu Chopra
Associate Professor
VSIT

Submitted by:

Pranjal Tiwari
12017704424

(Batch: 2024-2026)

Vivekananda Institute of Professional Studies – Technical Campus

Approved by AICTE, Accredited Grade "A++" Institution by NAAC, NBA
Accredited, Recognized under Section 2(f) by UGC, Affiliated to GGSIP University,
Recognized by Bar Council of India,
ISO 9001:2015 Certified

ACKNOWLEDGEMENT

It is my proud privilege to express my profound gratitude to the entire management of Vivekananda Institute of Professional Studies – Technical Campus and teachers of the institute for providing us with the opportunity to avail the excellent facilities and infrastructure. The knowledge and values inculcated have proved to be of immense help at the very start of my career. Special thanks to Hon'ble Founder, Vivekananda Institute of Professional Studies- Technical Campus, Delhi for having provided us an excellent infrastructure at VSIT.

I am grateful to Dr. Deepali Kamthania (Dean, VSIT), and Dr. Meenu Chopra for their astute guidance, constant encouragement and sincere support for this project work. Sincere thanks to all my family members, seniors and friends for their support and assistance throughout the project.

Pranjal Tiwari



Vivekananda Institute of Professional Studies – Technical Campus
(Affiliated to Guru Gobind Singh Indraprastha University)

CERTIFICATE

This is to certify that I Pranjal Tiwari, of MCA 2nd Semester from Vivekananda Institute of Professional Studies-Technical Campus, Delhi has presented this project work entitled “MediAI – An AI based Medical Diagnosis System ”, in partial fulfillment of the requirements for the award of the degree of Masters of Computer Applications under our supervision and guidance.

Date: 07-05-2025

Signature of the Guide

Dr. Meenu Chopra
Associate Professor

TABLE OF CONTENTS

S. No.	Title	Page No.	Teacher Sign
1.	Synopsis of the Project	1-2	
2.	Objective & Scope of the Project	3-4	
3.	Theoretical Background Definition of Problem	5-7	
4.	System Analysis & Design vis-a-vis User Requirements	8-16	
5.	Methodology adopted, System Implementation & Details of Hardware & Software used	17-18	
6.	System Maintenance & Evaluation	19-20	
7.	Detailed Life Cycle of the Project a. ERD, DFD, b. Input and Output Screen Design, c. Process involved, d. Methodology used testing, e. Test Report, Printout of the Report & CodeSheet	21-40	
8.	Coding and Screenshots of the project	41-61	
9.	Conclusion and Future Scope	62	
10.	References	63	

Synopsis

Project Title: MediAI – An AI based Medical Diagnosis System

Team Size: 1

Name of Member: Pranjali Tiwari

Technology Used: Machine Learning, Deep Learning, Image Analysis, Natural Language Processing, Large Language Models, Multimodal Data Integration, Predictive Analytics, Optical Character Recognition (OCR), Retrieval-Augmented Generation (RAG), Vision and Voice-based AI, Postman.

Abstract of the Project:

MediAI is an innovative AI-driven medical diagnosis system designed to enhance patient care through advanced diagnostic capabilities. This project leverages state-of-the-art machine learning algorithms to analyze medical data, providing healthcare professionals with accurate and timely diagnostic insights.

- 1. Data Integration and Management:** MediAI integrates various sources of medical data, including electronic health records, imaging data, and laboratory results, ensuring comprehensive patient profiles that facilitate accurate diagnosis.
- 2. AI-Powered Diagnostic Algorithms:** The system employs sophisticated AI algorithms trained on extensive medical datasets to identify patterns and anomalies in patient data, enabling precise diagnosis of a wide range of medical conditions.
- 3. User-Friendly Interface:** MediAI features an intuitive interface for healthcare providers, allowing easy access to diagnostic tools and patient information. This design promotes efficient workflows in clinical settings.
- 4. Real-Time Decision Support:** The platform offers real-time decision support by providing clinicians with recommendations based on the latest medical guidelines and research, enhancing clinical decision-making processes.
- 5. Security and Compliance:** MediAI prioritizes patient privacy and data security, adhering to healthcare regulations such as HIPAA. Robust encryption and secure access protocols protect sensitive patient information.
- 6. Scalability and Flexibility:** Built on a scalable architecture, MediAI can accommodate varying volumes of data and user demand, making it suitable for both clinics and hospitals.

7. Continuous Learning: The system incorporates feedback mechanisms that allow it to learn from new cases and outcomes, continuously improving its diagnostic accuracy over time.

8. Integration with Existing Systems: MediAI is designed to seamlessly integrate with existing healthcare IT systems, facilitating smooth adoption without disrupting current workflows.

9. Patient Engagement Tools: The platform includes features that enhance patient engagement, such as personalized health insights and educational resources about their conditions.

10. Research and Development Support: MediAI supports ongoing research by providing a platform for clinical trials and studies, contributing to the advancement of medical knowledge and practices.

MediAI aims to revolutionize the field of medical diagnostics by providing healthcare professionals with powerful tools that enhance accuracy, efficiency, and patient outcomes in clinical practice.

Main Report

(i) Objective of the Project

- **Enhanced Diagnostic Accuracy:**

Develop a system leveraging AI algorithms to provide healthcare professionals with precise diagnostic insights, enabling early and accurate detection of various medical conditions.

- **Increase Efficiency in Diagnostics:**

Streamline the diagnostic process by automating routine tasks and providing real-time insights, allowing healthcare professionals to focus on patient care rather than administrative duties.

- **Facilitate Early Disease Detection:**

Implement predictive analytics and OCR technology to analyze medical reports, prescriptions, and test results, enabling early detection of diseases such as thyroid disorders, diabetes, and cardiovascular diseases..

- **Efficient Decision Support:**

Offer real-time decision-making tools aligned with current medical guidelines, empowering clinicians to make evidence-based, timely treatment decisions.

- **Integrate RAG for Medical Knowledge Retrieval:**

Integrating Retrieval Augmented Generation enhances the system's ability to retrieve and process medical literature and past cases dynamically, providing more accurate and context-aware responses.

- **Develop a Multimodal Medical Assistant (Vision & Voice):**

Implement an interactive medical chatbot capable of processing visual inputs (X-rays, MRI scans, etc.) and responding through both text and voice, offering an intuitive and user-friendly experience for patients and medical professionals.

- **Continuous Algorithm Improvement:**

Incorporate continuous learning mechanisms into the AI to adapt and improve diagnostic capabilities and staying up to date with the latest medical knowledge.

(ii) Scope of the Project

- **OCR-Based Disease Prediction Expansion:**

Extend the OCR model to recognize and diagnose multiple medical conditions beyond thyroid disorders, making the system more versatile.

- **Medical Assistance with Vision & Voice Integration:**

Develop a Assistant capable of understanding medical images and responding through natural voice-based interactions, improving user engagement.

- **Integration of RAG for Advanced Medical Diagnosis:**

Enhance diagnostic capabilities using Retrieval-Augmented Generation to provide accurate medical insights based on a vast database of medical literature.

- **Improving Early Disease Detection Models:**

Train predictive models on a larger dataset to improve early diagnosis for conditions like diabetes, thyroid disorders, and cardiovascular diseases.

- **Enhanced AI-Based Image Analysis:**

Improve image-processing algorithms to analyze and diagnose conditions from medical imaging data like X-rays, MRIs, and CT scans.

(iii) Theoretical Background

The theoretical foundation of MediAI is rooted in the convergence of artificial intelligence (AI), medical diagnostics, and healthcare informatics. This project aims to leverage advanced AI methodologies to enhance diagnostic accuracy, efficiency, and personalization in medical practice.

Artificial Intelligence in Medical Diagnosis

- AI has become a pivotal element in the evolution of medical diagnostics, with its applications spanning various domains including image analysis, predictive modeling, and decision support systems. At its core, MediAI employs machine learning and deep learning algorithms to analyze complex medical data, which includes patient histories, imaging results, and genetic information. These algorithms are designed to detect patterns and correlations that may be imperceptible to human clinicians, thereby improving diagnostic outcomes significantly.

Data Integration and Processing

- MediAI's effectiveness hinges on its ability to process vast amounts of multimodal data. This includes not only traditional clinical data but also advanced imaging modalities such as MRI and CT scans. By employing sophisticated algorithms, MediAI can analyze these datasets rapidly, providing healthcare professionals with real-time insights that facilitate timely interventions. The system's architecture is designed to ensure seamless data integration from various sources, enhancing the comprehensiveness of the diagnostic process.

Enhancing Diagnostic Accuracy

- One of the primary goals of MediAI is to enhance diagnostic accuracy. Research indicates that AI algorithms can outperform traditional diagnostic methods by reducing human error and improving sensitivity in detecting conditions such as cancer or cardiovascular diseases. For instance, studies have shown that AI-driven tools can analyze mammograms with greater precision than human radiologists, leading to earlier detection and better patient outcomes.

Personalized Medicine

- MediAI also embraces the principles of personalized medicine by tailoring treatment plans based on individual patient data. By analyzing genetic profiles alongside clinical history, the system can recommend customized therapeutic strategies that align with each patient's unique health profile⁵. This approach not only optimizes treatment efficacy but also minimizes adverse effects by ensuring the interventions

Ethical Considerations and Human-AI Collaboration

- While the potential of AI in diagnostics is substantial, it is crucial to address ethical considerations surrounding its implementation. MediAI acknowledges the importance of maintaining a collaborative relationship between AI systems and healthcare professionals. The system is designed to augment human judgment rather than replace it; thus, clinicians remain integral to the diagnostic process, using AI generated insights as a tool for informed decision-making

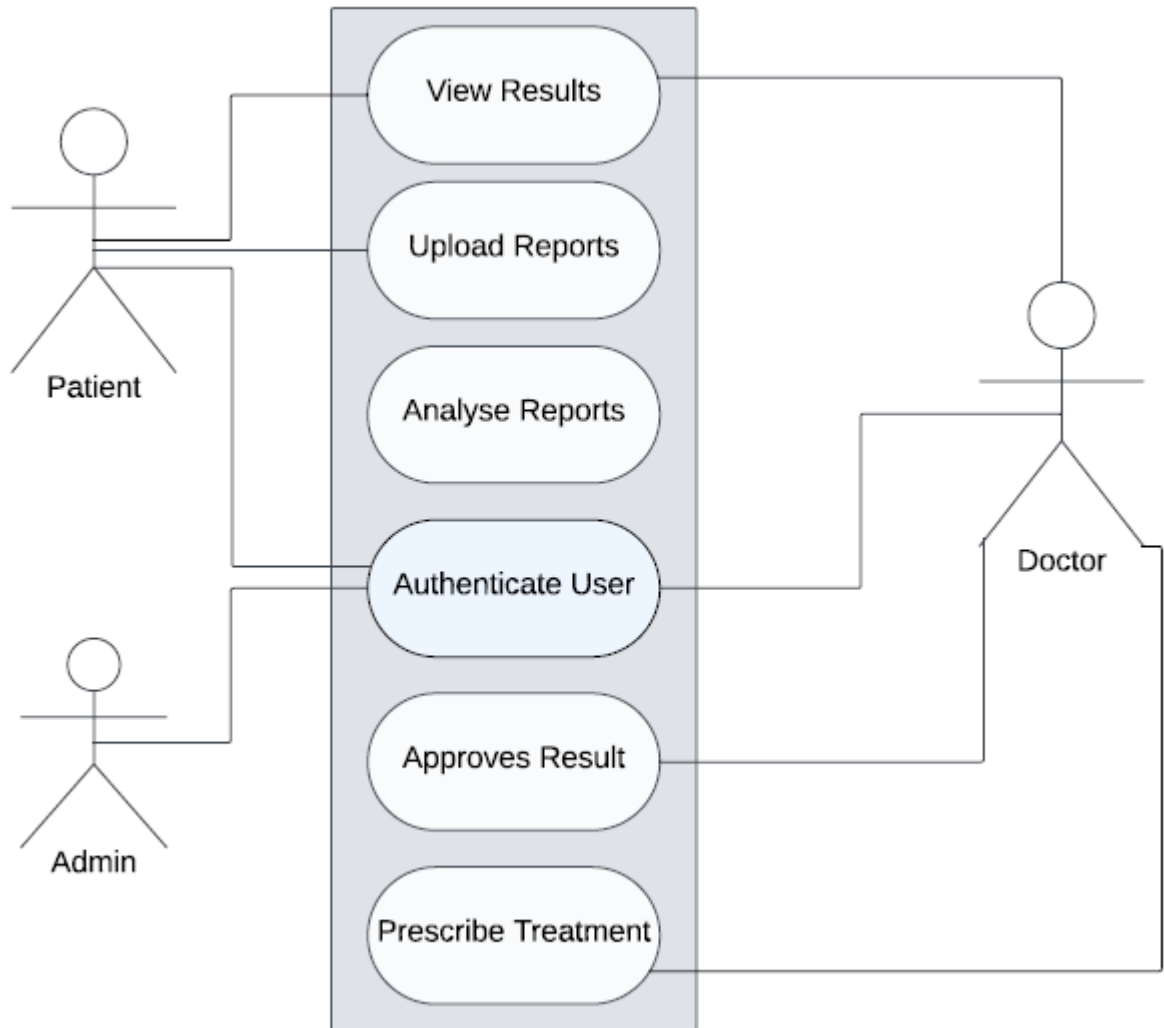
(iv) Definition of Problem

- The core problem addressed by the MediAI project is the significant gap in efficient, accurate, and accessible medical diagnostic systems that leverage artificial intelligence. While traditional diagnostic methods rely heavily on human expertise and are often subject to limitations such as human error, time constraints, and resource availability, the integration of AI technologies has the potential to enhance diagnostic accuracy and efficiency dramatically.
- MediAI seeks to overcome these challenges by providing a robust AI-based medical diagnosis system that utilizes advanced algorithms to analyze complex medical data. One of the primary issues tackled by MediAI is diagnostic accuracy. Traditional diagnostic processes can be slow and prone to errors, especially when interpreting medical images or analyzing patient histories. By employing machine learning and deep learning techniques, MediAI aims to improve diagnostic precision, enabling healthcare providers to make more informed decisions based on comprehensive data analysis.
- Additionally, accessibility is a crucial concern in healthcare diagnostics. Many existing systems are either too complex or not user-friendly, limiting their use by healthcare professionals in various settings. MediAI addresses this issue by offering a user-friendly interface that allows clinicians to access and interpret diagnostic information seamlessly, thus facilitating quicker decision-making processes.
- Moreover, scalability poses a challenge for many current diagnostic solutions. As demand for healthcare services increases, existing systems may struggle to handle large volumes of data efficiently. MediAI is designed with scalability in mind, ensuring that it can accommodate growing datasets and user demands without compromising performance.
- In summary, the problem statement for the MediAI project revolves around the need for an efficient, accurate, and accessible AI-driven medical diagnosis system that enhances diagnostic capabilities while addressing issues of usability and scalability. By integrating advanced AI technologies into the diagnostic process, MediAI aims to transform healthcare delivery, ultimately leading to improved patient outcomes and more effective treatment strategies.

(v) System Analysis & Design vis - a - vis User Requirements

1. System Analysis:

A. Use Case Diagram



1.1 Description

MediAI: Medical Diagnosis System is an AI-powered platform designed to assist healthcare professionals and patients by providing diagnostic insights through the analysis of medical data. The system facilitates accurate diagnoses, secure data management, and seamless interactions between users, including patients, doctors, and administrators.

1.2 Actors

- Patient: Uploads medical reports, views diagnostic results, and receives prescribed treatments.
- Doctor: Reviews analysis results, approves diagnoses, and prescribes treatments for patients.
- Admin: Authenticates users, monitors the system, and ensures secure access to data.

2. Flow of Events

2.1 Basic Flow

- Patient:
 - Uploads medical reports for analysis.
 - Views diagnostic results generated by the system.
 - Receives treatment prescriptions after approval by the doctor.
- Doctor:
 - Reviews diagnostic analysis from the system.
 - Approves or modifies results and prescribes treatments.
- Admin:
 - Authenticates users and manages roles.
 - Ensures secure data storage and compliance with healthcare regulations.

2.2 Alternative Flows

- Patient:
 - Requests clarifications or further explanations of diagnostic results.
 - Reports discrepancies or issues in the system.
- Doctor:
 - Overrides system suggestions based on clinical judgment.
 - Requests updates to AI models if results are inconsistent.
- Admin:
 - Generates usage and performance reports for analysis.
 - Updates system settings and resolves technical issues.

3. Special Requirements

- Ensure compliance with healthcare regulations like HIPAA to protect patient data.
- Provide robust authentication and encryption mechanisms for all users.
- Incorporate mechanisms for continuous updates to AI algorithms with new medical research.
- Maintain detailed audit logs for all user interactions to ensure accountability.

4. Pre-conditions

- The MediAI system is fully operational and accessible online.
- Patients, doctors, and admins must have valid user accounts with appropriate roles.
- Necessary medical reports and data should be available for upload and analysis.

5. Post-conditions

- Diagnostic results are securely stored and accessible only to authorized users.
- Approved diagnoses and treatments are logged and recorded for future reference.
- Administrative changes are reflected immediately across the system.

6. Extension Points

- Integration with EHR systems for streamlined data management and accessibility.
- Development of a mobile application to enhance accessibility for users on the go.
- - Implementation of a feedback system where users can report inaccuracies or suggest improvements for the diagnostic tool.

Related Use Cases

- **Input Symptoms:** Patients can enter their symptoms into the system for analysis.
- **Receive Diagnosis:** Patients receive feedback based on their input data.
- **Review Patient Records:** Doctors can access and review patient histories and diagnoses.
- **Manage Users:** Administrators can create, update, or delete user accounts within the system.
- This structured approach ensures that all interactions within the MediAI system are clearly defined, facilitating effective communication among stakeholders and supporting seamless operation of the medical diagnosis process.

2. System Interfaces

2.1 API Integration

- **External API Integration:**
Fetches data from medical knowledge bases (e.g., UMLS, PubMed), laboratory results from diagnostic labs, and patient medical histories from hospital EHR systems.
- **Exposed APIs:**
RESTful APIs and GraphQL endpoints provide MediAI's functionality for seamless integration with third-party applications.

2.2 User Interfaces

- **Web and Mobile Platforms:**
Responsive, user-friendly interface for both platforms.
- **Key Screens:**
 1. **Registration Screen:** For account creation (patients and healthcare)
 2. **Login Screen:** User authentication via secure mechanisms
 3. **Symptom Checker:** Inputs symptoms for diagnosis.
 4. **Diagnosis Results Dashboard:** Displays probable diagnoses with confidence levels, recommended next steps, and treatment suggestions.
 5. **Medical History Viewer:** Enables patients to review their health data and past interactions.
 6. **Admin Panel:** Allows system administrators to manage users, analyze logs, and retrain models.

2.3 Hardware Interfaces

- **Screen Resolution Requirements:**
Minimum resolution: 1024 x 768 for optimal usability.
- **Compatibility:**
Supports both standalone systems and cloud-based deployments.

2.4 Software Interfaces

Operating System Compatibility:

- Works with Windows, macOS, and Linux.

Database Management Systems:

- **PostgreSQL:** For structured data (e.g., patient demographics, medical history).
- **MongoDB:** For semi-structured data (e.g., symptom logs, diagnoses).

Development Tools:

- **Backend:** Django/Flask (Python) or Node.js.
- **Frontend:** React.js (web) and Flutter (mobile apps).
- **AI Models:** Diagnosis engine based on TensorFlow or PyTorch.

2.5 Communications Interfaces

- **Connectivity:**
Requires internet access for API communication and EHR integration.
- **Data Security:**
Transmits data securely using HTTPS with TLS encryption.

2.6 Memory Constraints

- **System Requirements:**
 - Minimum 4 GB RAM and 20 GB hard disk space for local installations.
 - Cloud-based deployments require scalable storage and memory, depending on user load.

2.7 Operations

- **Roles and Responsibilities:**
 - **Data Entry Operator:** Updates patient records and ensures data accuracy.
 - **Healthcare Professionals:** Use the system for diagnosis and treatment planning.
 - **System Administrator:** Manages backend servers, ML model updates, and database backups.
- **Database Backup:**
 - Automated daily backups with recovery tools provided for administrators.

2.8 Site Adaptation Requirements

- **Client-Side Requirements:**
 - Modern browsers (e.g., Chrome, Firefox) are required for web applications.
 - Mobile apps require iOS 11.0+ or Android 8.0+.
- **Scalability:**
 - Cloud-based infrastructure supports increased data and user traffic.
 - Supports edge computing for faster response times in remote areas.

3. User Requirements

User Requirements for MediAI: Medical Diagnosis System

3.1 External Interfaces

- User Interface:
 - MediAI provides a responsive interface accessible via web browsers and mobile applications.
 - Healthcare providers can access the system using hospital or clinic-based terminals.
- API Integration:
 - MediAI integrates with third-party APIs for retrieving medical knowledge, test results, and patient records.
- Data Management:
 - Administrators manage patient data and oversee system operations.
 - Data Entry Operators ensure accurate input of symptoms and medical history into the system.

3.2 Functional Requirements

3.2.1 User Registration

- Allows patients and healthcare professionals to create accounts by providing personal details, email addresses, and secure passwords.
- Includes email or SMS verification for identity confirmation.

3.2.2 User Login

- Enables registered users to log in using their email and password.
- Offers a "Forgot Password" feature for password recovery.
- Supports two-factor authentication for enhanced security.

3.2.3 Symptom Checker

- Allows users to input symptoms via text or voice.
- Uses AI to analyze symptoms and provide a list of probable diagnoses with confidence levels.

3.2.4 Medical History Integration

- Fetches and displays a patient's past medical records and prescriptions from EHR systems.
- Allows users to upload documents like lab reports or prescriptions.

3.2.5 Diagnosis Results Dashboard

- Displays diagnosis results, recommended tests, and potential treatments.
- Offers links to nearby labs or pharmacies for follow-up actions.

3.2.6 Test Recommendation

- Recommends diagnostic tests based on symptoms and patient history.
- Integrates with diagnostic lab systems for test scheduling and result retrieval.

3.3 Treatment Suggestions

3.3.1 Provides treatment suggestions based on diagnosis, including medication, lifestyle changes, or referrals to specialists.

3.3.2 Allows healthcare providers to customize or override AI suggestions.

3.4 User Feedback

3.4.1 Enables users to provide feedback on the accuracy and usefulness of diagnoses and recommendations.

3.4.2 Collects data for retraining AI models to improve accuracy.

3.5 Admin Dashboard

3.5.1 Provides administrators with tools to manage users, analyze system logs, and oversee AI model performance.

3.5.2 Supports model updates and data backup operations.

3.6 Notifications

3.6.1 Sends notifications for upcoming tests, follow-ups, or health tips.

3.6.2 Allows users to opt-in or opt-out of notifications.

4. Non-Functional Requirements

4.1 Performance

- Ensures quick processing of user inputs with response times under 2 seconds for common queries.
- Scales to handle increasing user loads without performance degradation.

4.2 Availability

- Ensures 99.9% uptime for uninterrupted access to the system.

4.3 Security

- Implements robust user authentication and role-based access control.
- Encrypts data in transit (HTTPS/TLS) and at rest (AES-256).
- Complies with healthcare regulations like HIPAA and GDPR.

4.4 Usability

- Provides an intuitive interface for both patients and healthcare professionals.
- Ensures compatibility across a wide range of devices, operating systems, and screen sizes.

4.5 Maintainability

- Allows administrators to easily update AI models, user roles, and system configurations.
- Logs system operations for troubleshooting and debugging.

4.6 Scalability

- Designed to support a growing number of users, patient records, and integrations with external systems.

4.7 Reliability

- Ensures accurate and consistent data processing, diagnosis generation, and result retrieval.
- Includes failover mechanisms to prevent downtime.

4.8 Extensibility

- Allows integration with future medical devices (e.g., wearables) and third-party health apps.
- Supports continuous addition of new features and datasets.

(vi) Methodology adopted, System Implementation & Details of Hardware & Software used, System Maintenance & Evaluation

1. Methodology Adopted

Methodology Adopted for MediAI: Medical Diagnosis System

The methodology adopted for MediAI, a Medical Diagnosis System, combines agile development principles with systematic approaches to ensure iterative processes and continuous improvement. Below is an outline of the methodology applied:

Agile Development

1. Agile Model

- Agile development is based on iterative development, breaking tasks into smaller iterations without long-term planning.
- Each iteration involves a full software development life cycle, including planning, requirements analysis, design, coding, and testing.
- Agile methods minimize project risks, reduce delivery time, and allow for flexibility in responding to changing requirements.

2. Phases of Agile Model

- Requirements gathering
- Designing requirements
- Construction/iteration
- Testing/Quality assurance
- Deployment
- Feedback

3. Waterfall Elements for Structured Planning

- **Initial Planning and Requirements Analysis:**
Thorough planning and requirements gathering to define scope, objectives, and user requirements.

4. **System Development Life Cycle (SDLC)**
 - Divides the project into distinct phases like planning, analysis, design, implementation, testing, and maintenance.
 - Allows for parallel development to optimize efficiency.
5. **Prototyping**
 - Develops prototypes for critical functionalities to gather early feedback from stakeholders and users.
 - Refines prototypes based on feedback to align with user expectations.
6. **Continuous Integration and Testing**
 - Regular integration of code into a shared repository for collaborative development.
 - Implements automated testing to ensure reliability and functionality.
7. **Collaboration Tools**
 - Utilizes version control systems like Git for effective collaboration and tracking changes.
 - Implements communication tools for seamless collaboration among team members.
8. **Data-Driven Decision Making**
 - Utilizes data analysis from diagnostic outcomes and user interactions to inform decision-making in design and feature enhancement.
9. **Scalability Considerations**
 - Develops a scalable architecture to accommodate potential increases in user traffic and platform expansion.
10. **User-Centric Design**
 - Actively seeks user feedback throughout development stages to align the final product with user expectations.
 - Conducts usability testing to address user experience challenges.
11. **Deployment and Maintenance**
 - Implements phased deployment for a smooth transition and addresses issues promptly.
 - Establishes processes for continuous improvement, bug fixing, feature updates, and adaptation to changing user needs.

Agile Model

1. Agile Development

- Agile development is based on iterative development, breaking tasks into smaller iterations without long-term planning.
- Each iteration involves a full software development life cycle, including planning, requirements analysis, design, coding, and testing.
- Agile methods minimize project risks, reduce delivery time, and allow for flexibility in responding to changing requirements.

2. Waterfall Elements for Structured Planning

- **Initial Planning and Requirements Analysis:**
Conducts thorough planning to define the system scope, objectives, and detailed user requirements.

3. System Development Life Cycle (SDLC)

- Divides the project into distinct phases such as planning, analysis, design, implementation, testing, and maintenance.
- Allows for parallel development to optimize efficiency and reduce delays.

4. Prototyping

- Develops prototypes for critical functionalities to gather early feedback from stakeholders and users.
- Refines prototypes based on feedback to ensure alignment with user expectations.

5. Continuous Integration and Testing

- Integrates code into a shared repository regularly to enable collaborative development.
- Utilizes automated testing to validate system reliability and functionality.

6. Collaboration Tools

- **Version Control:** Git and GitHub/GitLab are used for effective collaboration and tracking changes.
- **Communication Tools:** Tools like Slack, Microsoft Teams, and Jira facilitate seamless communication and task tracking among team members.

7. Data-Driven Decision Making

- Leverages data analysis from diagnostic outcomes and user interactions to enhance design and feature development.

8. Scalability Considerations

- Builds a scalable system architecture to handle increasing user traffic, patient data, and integration of new features.

9. User-Centric Design

- Seeks continuous user feedback during development to align the system with patient and healthcare provider expectations.
- Conducts usability testing to identify and address challenges in the user experience.

10. Deployment and Maintenance

- Implements phased deployment for smooth transitions and to address issues promptly.
- Establishes processes for ongoing bug fixes, feature updates, and system enhancements.

Hardware Details

- **Server Requirements:**
 - **Processor:** Intel Xeon or AMD EPYC, 2.5 GHz or higher
 - **Memory:** Minimum 16 GB RAM (scalable based on system load)
 - **Storage:** SSD with 500 GB capacity (expandable for growing data needs)
 - **Network:** High-speed internet connection with 1 Gbps bandwidth
- **Client-Side Requirements:**
 - **Device:** Desktop, laptop, or mobile with modern web browser support
 - **Display:** Minimum 1080p resolution
 - **Compatibility:** Android, iOS, Windows, macOS

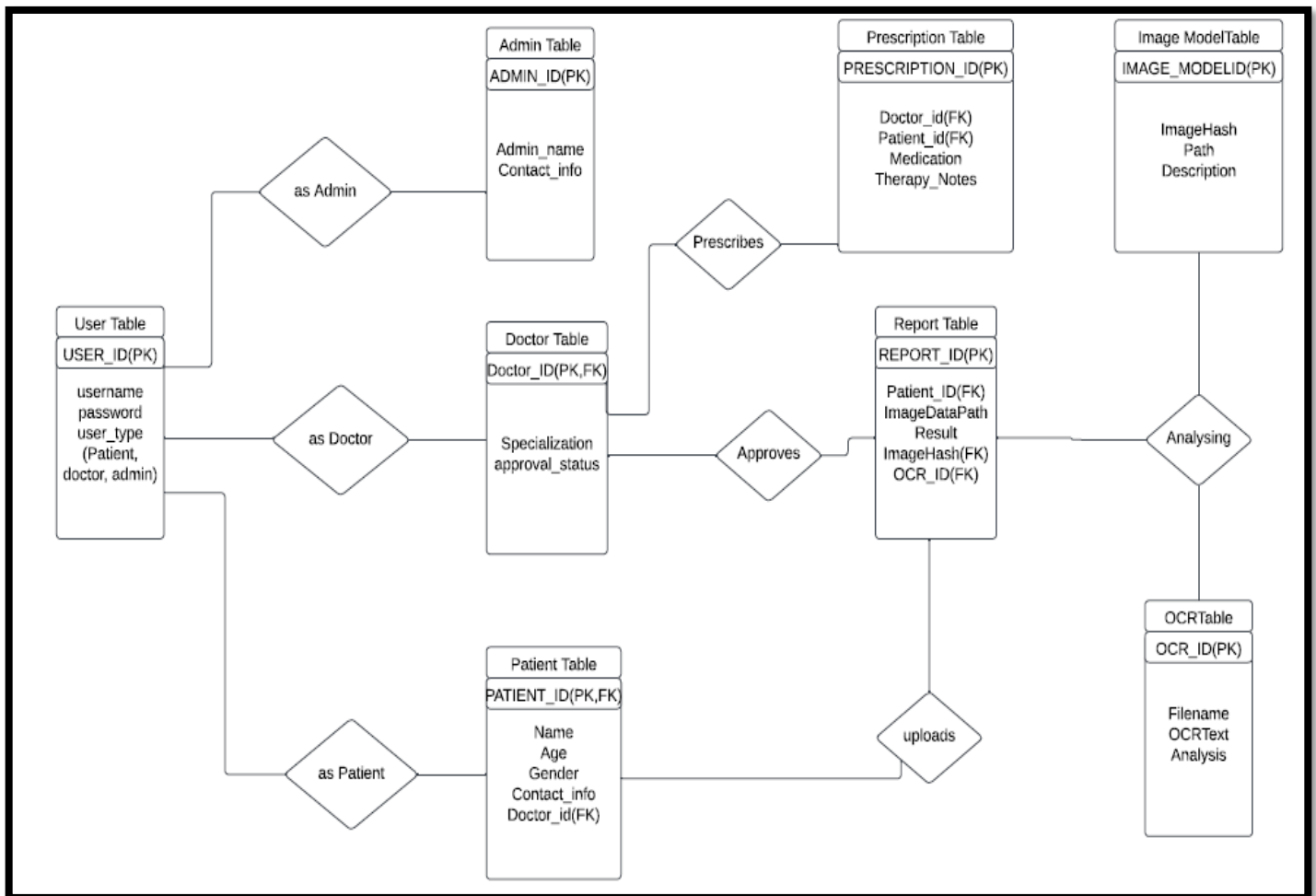
Software Details

- **Operating System:**
 - **Servers:** Linux-based systems (Ubuntu, CentOS) for stability and scalability
 - **Client Devices:** Cross-platform compatibility (Windows, macOS, Android, iOS)
 - **Development Tools:**
 - **Backend:** Python (Django/Flask)
 - **Frontend:** HTML5, CSS3, JavaScript (React.js/Angular)
 - **Database:** PostgreSQL or MongoDB for managing structured and unstructured data
 - **APIs and Libraries:**
 - OpenAI APIs for medical diagnosis and natural language processing
 - External EHR/EMR system integrations
 - **Testing Frameworks:**
 - Selenium, Pytest, and Postman for testing and quality assurance
 - **Version Control and CI/CD:**
 - Git for version control
 - Jenkins or GitHub Actions for continuous integration and deployment
-

1. Detailed Life Cycle of the Project

(a) ERD, DFD

1. ER Diagram



Description

The Entity-Relationship (ER) diagram provides a structured overview of the data model for the MediAI system, detailing the relationships between entities and their attributes. This description explains the entities, their key attributes, and the relationships among them.

1. Entities and Attributes

1.1 User Table

Primary Key: USER_ID

Attributes:

username: Stores the username for user login.

password: Stores the encrypted password for authentication.

user_type: Specifies the role of the user (Patient, Doctor, Admin).

Description:

This table manages all system users, with roles differentiated using user_type.

1.2 Patient Table

Primary Key: PATIENT_ID

Foreign Key: Doctor_ID (links the patient to their assigned doctor).

Attributes:

Name: Stores the patient's name.

Age: Records the age of the patient.

Gender: Specifies the gender of the patient.

Contact_info: Holds the patient's contact details.

Description:

This table contains specific details about patients, allowing the system to associate medical records and prescriptions with individual patients.

1.3 Doctor Table

Primary Key: Doctor_ID

Foreign Key: Linked to USER_ID in the User Table.

Attributes:

Specialization: Specifies the doctor's medical expertise.

approval_status: Indicates whether the doctor is approved for accessing and approving results.

Description:

This table manages data related to doctors, their specialization, and their ability to approve diagnoses.

1.4 Admin Table

Primary Key: Admin_ID

Attributes:

Admin_name: Stores the admin's name.

Contact_info: Contains the admin's contact details.

Description:

This table contains information about administrators who manage user accounts and monitor the system's operations.

1.5 Prescription Table

Primary Key: PRESCRIPTION_ID

Foreign Keys:

Doctor_ID: Links prescriptions to the doctor who issued them.

Patient_ID: Associates prescriptions with the respective patient.

Attributes:

Medication: Contains the prescribed medication details.

Therapy_Notes: Records any additional treatment instructions.

Description:

This table stores prescriptions generated for patients, associating them with the doctor and patient.

1.6 Report Table

Primary Key: REPORT_ID

Foreign Keys:

Patient_ID: Links the report to the respective patient.

ImageHash: Links to the Image Model Table for referencing uploaded images.

OCR_ID: Connects with the OCR Table for text analysis of uploaded reports.

Attributes:

ImageDataPath: Path to the stored image or report data.

Result: The diagnosis result generated by the AI model.

Description:

This table manages uploaded reports, storing results and linking them to patients for analysis and review.

1.7 Image Model Table

Primary Key: IMAGE_MODELID

Attributes:

ImageHash: Unique identifier for uploaded image data.

Path: Filepath for the stored image.

Description: Brief description of the image (e.g., X-ray, medical report).

Description:

This table stores information about the images used for analysis by the AI system.

1.8 OCR Table

Primary Key: OCR_ID

Attributes:

Filename: Name of the uploaded file.

OCRText: Extracted text from the uploaded report/image.

Analysis: Additional insights generated from the OCR text.

Description:

This table supports text extraction and analysis from uploaded medical reports, linking the results to the report table.

2. Relationships Between Entities

2.1 User Table Relationships

The USER_ID serves as the base identifier for users and connects:

To the Doctor Table as Doctor_ID for doctors.

To the Admin Table as Admin_ID for administrators.

To the Patient Table indirectly via their patient-specific data.

2.2 Patient-Doctor Relationship

Each patient in the Patient Table is associated with a doctor using the Doctor_ID. This ensures that the patient's reports and prescriptions are overseen by a specific medical professional.

2.3 Patient-Report Relationship

Patients upload reports, which are stored in the Report Table and associated with them through Patient_ID.

2.4 Report-Image Analysis Relationship

Each report can include an image file stored in the Image Model Table, linked via the ImageHash attribute. This allows the AI system to process medical images.

2.5 Report-OCR Relationship

The OCR_ID in the Report Table connects to the OCR Table, enabling analysis of text extracted from uploaded medical reports.

2.6 Doctor-Prescription Relationship

Doctors approve diagnostic results and issue prescriptions. The Prescription Table links these prescriptions to both the patient (Patient_ID) and the prescribing doctor (Doctor_ID).

2.7 Admin-System Management Relationship

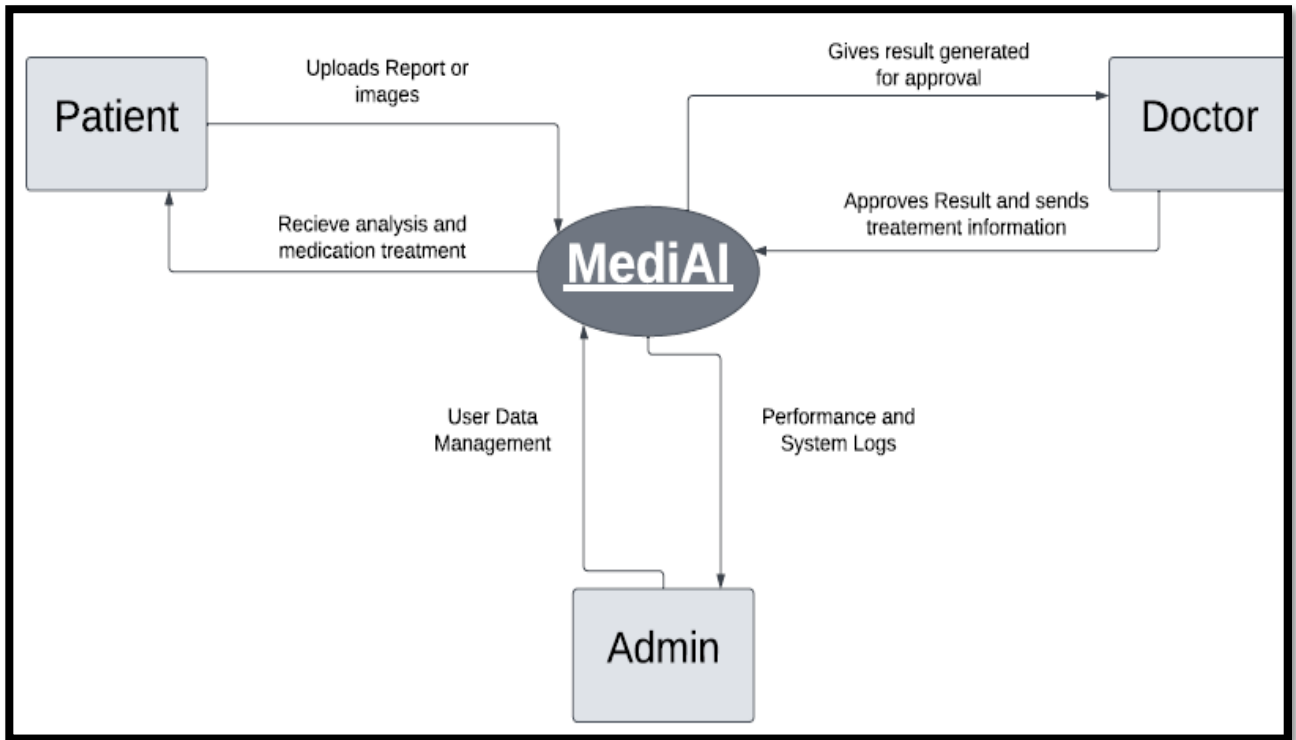
The admin manages system operations, overseeing users and ensuring the integrity of data in all tables. The Admin Table connects with the User Table indirectly to facilitate management roles.

3. Summary

The ER diagram for MediAI: Medical Diagnosis System illustrates a robust database structure designed to manage user roles, patient records, medical report analysis, and prescription generation. It ensures smooth interactions between patients, doctors, and administrators, with detailed data storage for images, OCR results, and prescriptions. This structure supports efficient AI-based medical diagnosis while maintaining data security and accessibility.

1. Data Flow Diagram

Level 0 DFD:



Description

The Level-0 Data Flow Diagram (DFD) for MediAI illustrates the high-level interactions between the main system (MediAI) and its key actors: Patient, Doctor, and Admin. Below is the detailed description of the components and their functions:

1. System Overview

MediAI serves as the central system for managing diagnostic processes, enabling interactions between patients, healthcare professionals, and administrators. It processes patient data, generates diagnostic results, and facilitates secure communication among actors.

2. Key Components and Interactions

2.1 Patient

- Inputs:

Uploads Reports or Images: The patient uploads medical reports or diagnostic images to the MediAI system for analysis.

- Outputs:

Receives Analysis and Medication Treatment: After the uploaded data is processed and approved by a doctor, the patient receives the analysis results and prescribed treatments.

2.2 Doctor

- Inputs:

Receives Results for Approval: MediAI generates diagnostic results based on the patient's medical data and shares them with the doctor for review.

- Outputs:

Approves Results and Sends Treatment Information: The doctor validates the diagnostic results and provides treatment recommendations or prescriptions, which are then sent to the patient.

2.3 Admin

- Inputs:

User Data Management: The admin manages user accounts, ensuring that only authorized users (patients and doctors) have access to the system.

- Outputs:

Performance and System Logs: The admin monitors system performance, reviews activity logs, and ensures the system's reliability and compliance with healthcare regulations.

3. Data Flow Description

1. Patient to MediAI:

- Patients upload reports or medical images for diagnosis.
- MediAI processes the data using AI algorithms to generate diagnostic suggestions.

2. MediAI to Doctor:

- The diagnostic results generated by MediAI are forwarded to the doctor for validation and approval.

3. Doctor to MediAI:

- The doctor reviews the results and sends back approved diagnostic findings along with treatment plans.

4. MediAI to Patient:

- The system provides patients with the approved results and prescribed treatments.

5. Admin to MediAI:

- Admins manage user data, monitor system activity, and ensure security compliance.

6. MediAI to Admin:

- The system generates performance reports and system logs for administrative monitoring and auditing.

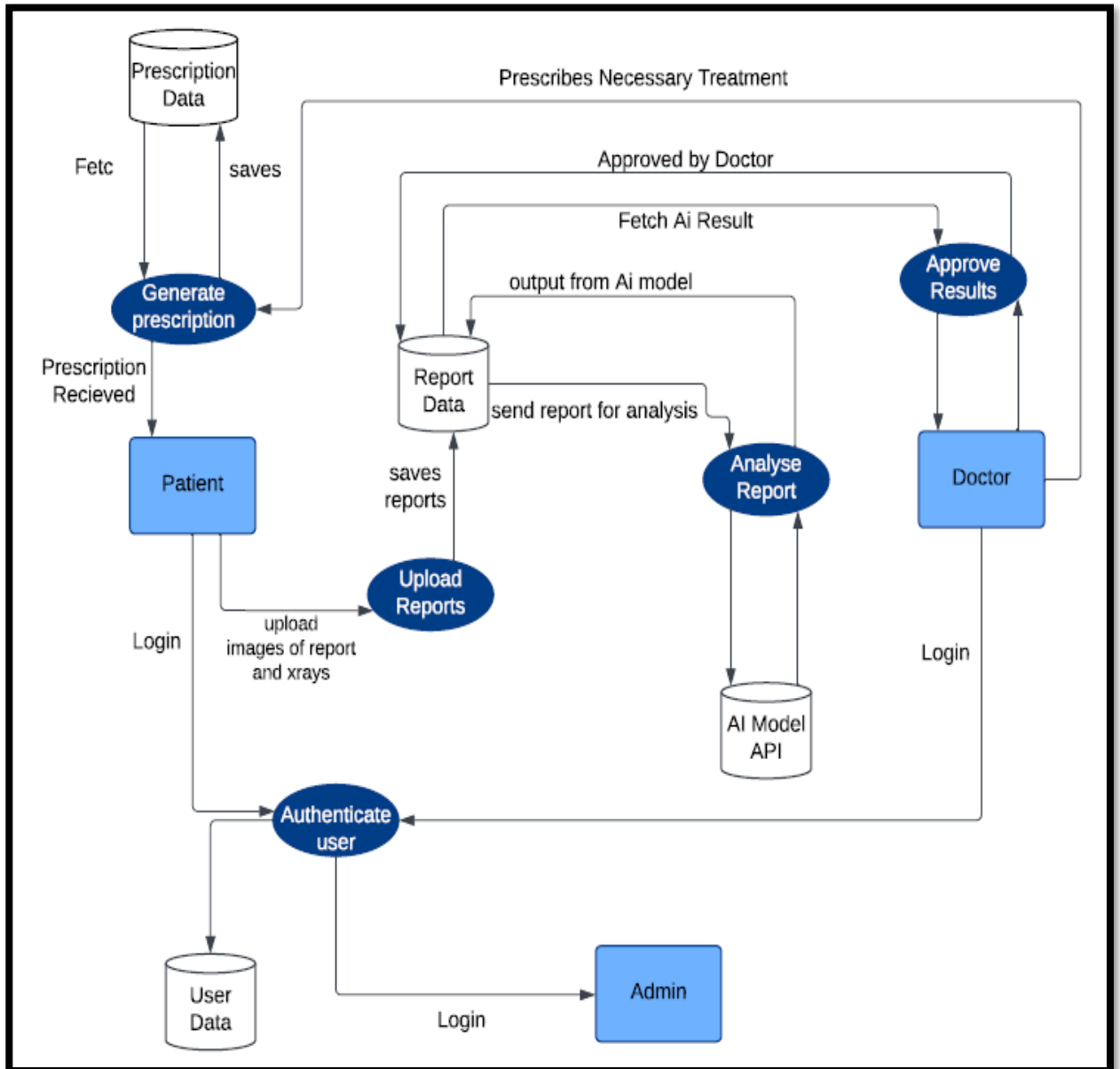
4. System Responsibilities

- **Data Processing:** MediAI processes uploaded medical data (e.g., reports and images) using AI to generate accurate diagnostic results.
- **Approval Workflow:** MediAI ensures that diagnostic results are reviewed and approved by doctors before being shared with patients.
- **User Management:** Admins manage authentication, ensuring secure and reliable system access for all users.
- **Compliance and Monitoring:** MediAI provides logs and performance metrics for system transparency and compliance with medical regulations.

5. Summary

The Level-0 DFD captures the core functionalities of MediAI by illustrating the flow of data between actors and the system. It highlights MediAI's role in streamlining medical diagnostics, ensuring accurate results, and enabling seamless communication between patients, doctors, and administrators. This foundational diagram sets the stage for further detailed breakdowns at subsequent levels of DFD.

Level 1 DFD:



Description

The Level-1 Data Flow Diagram (DFD) for MediAI provides a detailed view of the processes, data flows, and interactions between actors (Patient, Doctor, Admin) and the system components. This diagram elaborates on the flow of information from data input to diagnostic output and prescription generation.

1. System Overview

MediAI facilitates the diagnostic process by analyzing medical reports, fetching AI-based results, and generating treatment prescriptions. It ensures seamless communication between patients, doctors, and admins, while maintaining secure user authentication and data storage.

2. Key Processes and Data Stores

2.1 Authenticate User

- Inputs:
 - Login credentials from the Patient, Doctor, and Admin.
- Outputs:
 - Authenticated user sessions.
- Data Store:
 - User Data: Stores user credentials and roles for authentication.
- Description:
 - MediAI validates user credentials to allow secure access. Each user type (Patient, Doctor, Admin) is granted role-specific permissions.

2.2 Upload Reports

- Inputs:
 - Patients upload medical reports, such as images and X-rays.
- Outputs:
 - Reports saved in the Report Data store.
- Data Store:
 - Report Data: Contains all uploaded reports for processing and future reference.
- Description:
 - Patients upload reports, which are securely stored for analysis. These reports are essential for AI-based diagnosis.

2.3 Analyze Report

- Inputs:
 - Reports retrieved from the Report Data store.
- Outputs:
 - Diagnostic results generated by the AI Model API.
- Data Store:
 - AI Model API: Contains the trained AI algorithms used to analyze medical reports.
- Description:
 - MediAI processes the uploaded reports using advanced AI algorithms. The system identifies patterns and generates preliminary diagnostic suggestions.

2.4 Approve Results

- Inputs:
 - Diagnostic results generated by the AI Model API.
- Outputs:
 - Approved diagnostic results, including recommendations for treatment.
- Description:
 - Doctors review and approve the diagnostic results. They validate the AI's findings and provide necessary recommendations before sharing with the patient.

2.5 Generate Prescription

- Inputs:
 - Approved diagnostic results and recommendations from the Doctor.
- Outputs:
 - Treatment plans or prescriptions sent to the patient.
- Data Store:
 - Prescription Data: Stores generated prescriptions for record-keeping and future access.
- Description:
 - Based on the approved results, MediAI generates a prescription that is shared with the patient for treatment. The system ensures that all prescriptions are securely logged.

3. Actors and Their Interactions

3.1 Patient

- Inputs:
 - Logs into the system to upload reports.
- Outputs:
 - Receives analysis results and prescribed treatments.
- Description:
 - Patients interact with the system to upload medical data and receive personalized diagnostic insights and treatments.

3.2 Doctor

- Inputs:
 - Logs into the system to fetch AI results for review.
- Outputs:
 - Approves results and prescribes treatments.
- Description:
 - Doctors play a crucial role in validating AI-generated diagnoses and recommending appropriate treatments.

3.3 Admin

- Inputs:
 - Logs into the system for monitoring and configuration.
- Outputs:
 - Manages user data and oversees system performance.
- Description:
 - Admins authenticate users, manage system settings, and monitor overall performance to ensure reliability and compliance.

4. Data Flows and Interactions

1. Patient to MediAI:
 - Patients upload reports, which are stored in the Report Data store and sent for analysis.
2. MediAI to Doctor:
 - Diagnostic results from the AI Model API are sent to the doctor for review and approval.
3. Doctor to MediAI:
 - Approved results and treatment recommendations are sent back to the system for prescription generation.
4. MediAI to Patient:
 - Patients receive diagnostic results and treatment prescriptions.
5. Admin to MediAI:
 - Admins oversee user management and ensure secure access.

5. Summary

This Level-1 DFD of MediAI highlights the step-by-step flow of information, from patient data submission to prescription delivery. It ensures efficient collaboration between patients, doctors, and administrators while leveraging AI-powered diagnostic capabilities. The system emphasizes secure data storage, user authentication, and a streamlined workflow to enhance healthcare outcomes.

(a) Process involved

1. Initiation

- **Define Project Scope:**
Identify objectives such as accurate medical diagnosis, ease of use for medical professionals, and integration with existing healthcare systems.
- **Stakeholder Identification:**
Engage healthcare providers, IT specialists, patients, and regulatory authorities.
- **Preliminary Project Plan:**
Develop an initial project plan outlining goals, timelines, and expected deliverables.
- **Approval:**
Obtain approval from healthcare institutions and regulatory bodies to proceed with development.

2. Planning

- **Detailed Project Plan:**
Prepare a comprehensive project plan with tasks, milestones, and allocated resources.
- **Risk Assessment:**
Identify potential risks including data security, compliance with healthcare regulations, and system accuracy. Develop mitigation strategies.
- **Team Roles and Responsibilities:**
Define specific roles for healthcare experts, developers, and testers.

3. Design

- **System Architecture:**
Create a robust and scalable architecture to handle medical data and diagnostic models.
- **User Interface Design:**
Design intuitive and accessible interfaces for healthcare providers and patients, including wireframes and prototypes.
- **Database Design:**
Develop a secure schema to store patient data, medical histories, and diagnostic results.

4. Implementation

- **Front-End Development:**
Build a user-friendly interface using modern frameworks like React.js or Angular.
- **Back-End Development:**
Develop back-end services using Python (Django/Flask) to manage data and implement diagnosis logic.
- **Data Integration:**
Integrate external data sources such as Electronic Health Records (EHR) and medical databases.
- **AI & ML Models:**
Implement machine learning algorithms for diagnosis and treatment recommendations.
- **Regulatory Compliance:**
Ensure adherence to healthcare standards such as HIPAA for data privacy.

5. Testing

- **Unit Testing:**
Validate individual components, including diagnostic algorithms and UI features.
- **Integration Testing:**
Ensure seamless interaction between front-end, back-end, and data sources.
- **Automated Testing:**
Use automated tools to verify system reliability under varying conditions.
- **User Acceptance Testing (UAT):**
Involve medical professionals in testing to ensure the system meets their expectations and workflows.

6. Deployment

- **Phased Deployment:**
Gradually roll out the system, starting with pilot programs in selected healthcare institutions.
- **System Monitoring:**
Continuously monitor the system for performance and accuracy in live environments.
- **Accessibility Check:**
Ensure the system functions optimally across devices and for users with

accessibility needs.

7. Operation and Maintenance

- **Transition to Operation:**
Make the system available for regular use by medical professionals and institutions.
- **Monitoring:**
Monitor system performance, user feedback, and compliance with regulations.
- **Regular Updates:**
Address bugs, improve diagnostic algorithms, and introduce new features based on feedback.
- **Ongoing Enhancements:**
Adapt the system to evolving medical practices and technologies.

8. Evaluation

- **Assessing Success:**
Measure project success against objectives like diagnostic accuracy, user satisfaction, and system reliability.
- **Performance Review:**
Review adherence to timelines, budget, and quality standards.
- **User Feedback:**
Collect input from healthcare providers and patients to guide future improvements.

9. Scaling (Future Phase)

- **User Traffic Evaluation:**
Monitor usage patterns to determine the need for scaling up infrastructure.
- **Scalability Enhancements:**
Enhance system capacity to accommodate a growing user base and expanding datasets.
- **Future Features:**
Plan additional features such as mobile app integration and advanced AI diagnostics.

10. Closure

- **Documenting Lessons Learned:**
Compile a comprehensive report of challenges faced and solutions implemented.
- **Archive Documentation:**
Store project documentation, code repositories, and test results securely.
- **Communication:**
Inform stakeholders of project completion and thank them for their collaboration.

(b) Methodology used in testing

The testing methodology for MediAI: Medical Diagnosis System involves a multi-phase approach to ensure the system's functionality, reliability, security, and compliance with medical standards. Below is a breakdown of the methodology used for testing:

1. Types of Testing

1.1 Unit Testing

- Objective: To test individual components of the system in isolation, ensuring that each function or module performs as expected.
- Scope:
 - Testing of AI algorithms for correct analysis of medical reports.
 - Testing individual functions such as prescription generation, report uploading, and result approval.
 - Unit tests will also be written for backend services (e.g., database queries, user authentication).
- Tools:
 - Python's `unittest` framework or `pytest` for backend logic.
 - Mocking libraries for testing AI model functions without needing real inputs.

1.2 Integration Testing

- Objective: To validate that different modules of the system work together as expected when integrated.
- Scope:
 - Ensuring that the AI model API integrates seamlessly with the report upload and analysis functionality.
 - Verifying that the user authentication and authorization system functions correctly across user roles (doctor, patient, admin).
 - Testing interactions between the Patient Table, Report Table, OCR Table, and Prescription Table to ensure that data flows smoothly.
- Tools:
 - Postman or Swagger for API integration testing.
 - Selenium for end-to-end UI tests

1.3 Functional Testing

- Objective: To validate that the system meets the functional requirements outlined in the specifications.

- Scope:
 - Ensuring that patients can upload reports and images, receive results, and generate prescriptions.
 - Validating that doctors can approve results and prescribe necessary treatments.
 - Confirming that admins can manage users and oversee the system's performance.
- Tools:
 - Functional testing tools like TestComplete or Katalon Studio.
 - Manual testing to check system behavior under normal operation.

1.4 User Acceptance Testing (UAT)

- Objective: To verify that the system meets the needs and expectations of end users, including patients, doctors, and administrators.
- Scope:
 - Testing the user interface (UI) to ensure ease of use for patients and doctors.
 - Verifying that the system's medical diagnosis suggestions align with real-world medical practices.
 - Gathering feedback from medical professionals on the accuracy and relevance of the AI-generated results.
- Tools:
 - User feedback surveys.
 - Manual UI testing.

1.5 Security Testing

- Objective: To ensure that the system is secure from potential threats, protecting sensitive patient data and ensuring compliance with healthcare regulations.
- Scope:
 - Ensuring secure data storage and transmission, especially for sensitive medical records.
 - Verifying that only authorized users (admin, doctors, patients) can access specific features based on their role.
 - Testing for common vulnerabilities (e.g., SQL injection, cross-site scripting, etc.).
 - Ensuring compliance with healthcare data protection laws such as HIPAA (Health Insurance Portability and Accountability Act).
- Tools:

- OWASP ZAP or Burp Suite for vulnerability scanning.
- Static application security testing (SAST) and dynamic application security testing (DAST) tools.

1.6 Performance Testing

- Objective: To ensure that the system can handle the expected load and deliver a responsive user experience.
- Scope:
 - Testing system behavior under load (multiple users uploading reports, doctors approving results simultaneously).
 - Measuring response times for AI model predictions and report generation.
 - Evaluating system performance during peak usage times.
- Tools:
 - JMeter or LoadRunner for load testing.
 - Google Lighthouse for frontend performance testing.

1.7 Regression Testing

- Objective: To ensure that new updates or changes to the system do not introduce new bugs or break existing functionality.
- Scope:
 - Re-running previously executed test cases after changes to the codebase or functionality.
 - Ensuring that fixes or new features do not negatively affect the system's previous capabilities.
- Tools:
 - Automated testing tools (e.g., Selenium, pytest) to quickly re-test critical functionalities.

1.8 Compliance Testing

- Objective: To ensure that the system adheres to relevant healthcare regulations and medical standards.
- Scope:
 - Verifying that the system handles patient data in compliance with laws like HIPAA (USA) or GDPR (EU) for data privacy and security.
 - Ensuring that the AI system follows ethical guidelines for medical diagnoses.

- Confirming the accuracy of AI-generated diagnosis based on historical medical data and expert reviews.
- Tools:
 - Manual audits and compliance checklists.
 - Third-party compliance testing and certification services.

2. Testing Phases

2.1 Test Planning

- Define the scope of testing, objectives, and resources needed.
- Develop test cases, scenarios, and user stories.
- Identify potential risks and mitigation strategies.

2.2 Test Execution

- Conduct the planned tests and document results.
- Report any issues, bugs, or discrepancies found during testing.

2.3 Bug Reporting and Fixing

- After detecting bugs, document them in a bug-tracking tool (e.g., JIRA).
- Developers will fix the issues, and re-testing will occur to ensure the fix works.

2.4 Test Closure

- Summarize testing results, including any unresolved issues and pending tasks.
- Provide a test summary report to stakeholders for review.

3. Test Metrics and Reporting

- Test Coverage: Track the percentage of code and features tested.
- Defect Density: Measure the number of defects found per unit of code.
- Test Case Execution: Report the number of test cases executed and passed.
- Response Time and Throughput: Track system performance during tests.
- User Feedback: Collect feedback from users about the system's usability and accuracy.

4. Automation in Testing

Automated testing is implemented in the MediAI system for repetitive and high-volume tasks such as regression testing and performance testing. This speeds up the testing process and ensures consistent results.

5. Conclusion

The testing methodology for MediAI follows industry best practices, ensuring the system is secure, reliable, and effective. By combining manual and automated testing, the system can be evaluated for correctness, performance, and user experience, while also ensuring compliance with healthcare regulations. This methodology guarantees that MediAI can deliver high-quality, trustworthy, and accurate medical diagnoses.

Coding and Screenshots of the project

Code :

Phase-2/ocr/app.py

```
import os
import pickle
import numpy as np
import pandas as pd
from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename
import pytesseract
from PIL import Image
from database import init_db, insert_report, get_report, get_all_reports, get_connection
from utils import process_image, extract_lab_values, check_sufficient_values

# Initialize Flask app
app = Flask(__name__)
UPLOAD_FOLDER = 'uploads'
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

# Initialize database
init_db()

# Load trained model, label encoder, and scaler
with open("model/disease_model.pkl", "rb") as f:
    model = pickle.load(f)

with open("model/label_encoder.pkl", "rb") as f:
    label_encoder = pickle.load(f)

with open("model/scaler.pkl", "rb") as f:
    scaler = pickle.load(f)

# Lab value mapping for diseases
LAB_KEYS = ["TSH", "T3", "T4", "Glucose", "HbA1c", "Hemoglobin", "RBC", "Creatinine", "Urea"]

# Function to check allowed file types
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# Predict disease
def predict_disease(lab_values):
    values = list(lab_values.values())
```

```

if None in values:
    print("[WARNING] Insufficient lab values:")
    for key, value in lab_values.items():
        print(f"{key}: {value}")
    return "Insufficient lab values for prediction"

df = pd.DataFrame([values], columns=LAB_KEYS)
print("[INFO] Lab values as DataFrame:")
print(df)

df_scaled = scaler.transform(df)
prediction = model.predict(df_scaled)
return label_encoder.inverse_transform(prediction)[0]

# Upload Endpoint
@app.route('/upload', methods=['POST'])
def upload_report():
    if 'file' not in request.files:
        return jsonify({'error': 'No file part'}), 400
    file = request.files['file']
    if file.filename == "":
        return jsonify({'error': 'No selected file'}), 400

    if file and allowed_file(file.filename):
        filename = secure_filename(file.filename)
        file_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        file.save(file_path)

        print("[INFO] Image saved to:", file_path)

        ocr_text = process_image(file_path)
        print("\n=== OCR Text Extracted ===\n")
        print(ocr_text)

        lab_values_full = extract_lab_values(ocr_text)
        # Only keep keys required by the model
        lab_values = {k: lab_values_full.get(k) for k in LAB_KEYS}

        print("\n=== Extracted Lab Values ===\n")
        for k, v in lab_values.items():
            print(f"{k}: {v}")

        if not check_sufficient_values(lab_values, min_required=6):
            prediction = "Insufficient lab values for prediction :-"
        else:
            prediction = predict_disease(lab_values)

# Predict only if sufficient
prediction = predict_disease(lab_values)

```

```

conn=get_connection()
report_id = insert_report(conn, filename, ocr_text, prediction, lab_values)
conn.close

return jsonify({
    'message': 'Report uploaded successfully',
    'report_id': report_id,
    'disease_prediction': prediction
}), 201
return jsonify({'error': 'Invalid file format'}), 400

# Get Report by ID
@app.route('/report/<int:report_id>', methods=['GET'])
def get_report_details(report_id):
    report = get_report(report_id)
    if report:
        filename, ocr_text, lab_values, analysis = report
        return jsonify({
            'filename': filename,
            'ocr_text': ocr_text,
            'lab_values': lab_values,
            'analysis': analysis
        })
    return jsonify({'error': 'Report not found'}), 404

# Get All Reports
@app.route('/reports', methods=['GET'])
def get_all_report_details():
    reports = get_all_reports()
    return jsonify([
        {
            'id': report[0],
            'filename': report[1],
            'analysis': report[2]
        } for report in reports])

if __name__ == '__main__':
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    app.run(debug=True)

```


Phase-2/ocr/ train_model.py

```
import os
import pandas as pd
import numpy as np
import pickle
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Paths
MODEL_DIR = "model"
DATASET_PATH = os.path.join(MODEL_DIR, "updated_medical_lab_dataset.csv")
MODEL_PATH = os.path.join(MODEL_DIR, "disease_model.pkl")
ENCODER_PATH = os.path.join(MODEL_DIR, "label_encoder.pkl")
SCALER_PATH = os.path.join(MODEL_DIR, "scaler.pkl")

# Ensure the model directory exists
os.makedirs(MODEL_DIR, exist_ok=True)

# Load dataset
df = pd.read_csv(DATASET_PATH)

# Feature columns (lab test values)
feature_cols = ["TSH", "T3", "T4", "Glucose", "HbA1c", "Hemoglobin", "RBC", "Creatinine", "Urea"]
X = df[feature_cols]
y = df["Disease_Label"]

# Encode disease labels
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Standardize lab values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.2, random_state=42)

# Train RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate model
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

```

print(f"Model Accuracy: {accuracy * 100:.2f}%")

# Save model and utilities
with open(MODEL_PATH, "wb") as f:
    pickle.dump(model, f)

with open(ENCODER_PATH, "wb") as f:
    pickle.dump(label_encoder, f)

with open(SCALER_PATH, "wb") as f:
    pickle.dump(scaler, f)

print("Model, Label Encoder, and Scaler saved successfully.")

```

Phase-2/ocr/utils.py

```

import pytesseract
import re
import difflib
from PIL import Image

# Define expected test names and their regex patterns
EXPECTED_KEYS = {
    'TSH': r'TSH[:\s]*([\d.,]+)',
    'T3': r'T3[:\s]*([\d.,]+)',
    'T4': r'T4[:\s]*([\d.,]+)',
    'Glucose': r'Glucose[:\s]*([\d.,]+)',
    'HbA1c': r'HbA1c[:\s]*([\d.,]+)',
    'Hemoglobin': r'Hemoglobin[:\s]*([\d.,]+)',
    'RBC': r'RBC[:\s]*([\d.,]+)',
    'Creatinine': r'Creatinine[:\s]*([\d.,]+)',
    'Urea': r'Urea[:\s]*([\d.,]+)'
}

def clean_ocr_text(raw_text):
    """Clean and normalize OCR output for better extraction."""
    text = raw_text.replace('|', ':').replace('{', '').replace('}', '')

```

```

text = re.sub(r'\s{2,}', ' ', text) # Collapse multiple spaces
return text

def fuzzy_find(text, keyword, cutoff=0.6):
    """Find closest match in OCR output for fuzzy keyword matching."""
    words = re.findall(r'\w+', text.lower()) # normalize text
    matches = difflib.get_close_matches(keyword.lower(), words, n=1, cutoff=cutoff)
    return matches[0] if matches else None

def extract_lab_values(text):
    """Extract lab test values using regex and fuzzy matching, supporting comma/period decimals."""
    results = {}
    text = text.replace(",", ".") # Ensure uniform decimal format

    for key, pattern in EXPECTED_KEYS.items():
        match = re.search(pattern, text, re.IGNORECASE)
        if match:
            results[key] = match.group(1).strip()
        else:
            fuzzy_key = fuzzy_find(text, key)
            if fuzzy_key:
                fallback_pattern = rf"{re.escape(fuzzy_key)}[^\d]*([\d.]+)"
                match = re.search(fallback_pattern, text, re.IGNORECASE)
                results[key] = match.group(1).strip() if match else None
            else:
                results[key] = None

    print("\n=== Extracted Lab Values ===")
    for k, v in results.items():
        print(f"{k}: {v}")
    return results

def process_image(image_path):
    """Perform OCR on image and return extracted & cleaned text."""
    print("[INFO] Processing image:", image_path)
    try:
        img = Image.open(image_path)
        text = pytesseract.image_to_string(img)

        print("\n=== OCR Text Extracted ===\n")
        print(text)

        cleaned_text = clean_ocr_text(text)
        return cleaned_text

```

```
except Exception as e:
    print("[ERROR] Failed to process image:", e)
    return ""
```

```
def check_sufficient_values(lab_values, min_required=5):
    """Check if at least `min_required` lab values are present (not None)."""
    present_values = [v for v in lab_values.values() if v is not None]
    return len(present_values) >= min_required
```

Phase-2/ocr/database.py

```
import sqlite3
import os
import json

DB_PATH = 'OCR_reports.db'

# --- Database Connection ---
def get_connection():
    return sqlite3.connect(DB_PATH)

# --- Initialize Database Table ---
def init_db():
    """Create the reports table if it doesn't exist."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS reports (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            filename TEXT NOT NULL,
            ocr_text TEXT,
            lab_values TEXT,
            analysis TEXT
        )
    """)
    conn.commit()
    conn.close()

# --- Insert New Report ---
def insert_report(conn, filename, ocr_text, analysis, lab_values):
    """
    Inserts a new report into the database.
    :param conn: sqlite3 connection object
    :param filename: Name of the file
    :param ocr_text: Extracted text from the report
    :param analysis: Predicted disease or message
    :param lab_values: Dictionary of lab values
    """
```

```

:return: report ID
"""
cursor = conn.cursor()
cursor.execute("""
    INSERT INTO reports (filename, ocr_text, lab_values, analysis)
    VALUES (?, ?, ?, ?)
""", (
    filename,
    ocr_text,
    json.dumps(lab_values),
    analysis
))
conn.commit()
report_id = cursor.lastrowid
print(f"[INFO] Report saved with ID: {report_id}")
return report_id

# --- Fetch Single Report ---
def get_report(report_id):
    """Retrieve full report by ID."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT filename, ocr_text, lab_values, analysis FROM reports WHERE id = ?',
        (report_id,))
    report = cursor.fetchone()
    conn.close()
    return report

# --- Fetch All Reports (Summary) ---
def get_all_reports():
    """Return ID, filename, and analysis for all reports."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT id, filename, analysis FROM reports')
    reports = cursor.fetchall()
    conn.close()
    return reports

```

Phase-2/vision+voice/brain.py

```

# step 1:setup groq api key |
import os

GROQ_API_KEY=os.environ.get("GROQ_API_KEY")
# step 2: convert image to requiredformat
import base64

```

```

# image_path="images/acne.jpg"
def encode_image(image_path):
    image_file=open(image_path,"rb")
    return base64.b64encode(image_file.read()).decode('utf-8')

# \ step 3 : setup Multimodal LLM
from groq import Groq
# if not GROQ_API_KEY:
#     raise ValueError("GROQ_API_KEY is not set. Please provide your API key.")

# query="Is there something wrong with my face?"
# model="llama-3.2-90b-vision-preview"
def analyze_image_with_query(query,model,encoded_image):

    client=Groq(api_key=GROQ_API_KEY)

    messages=[
        {
            "role":"user",
            "content":[
                {
                    "type":"text",
                    "text":query
                },
                {
                    "type":"image_url",
                    "image_url":{"
                        "url":f"data:image/jpeg;base64,{encoded_image}"
                    },
                },
            ],
        }
    ]
    chat_completion=client.chat.completions.create(
        messages=messages,
        model=model
    )
    return chat_completion.choices[0].message.content

```

Phase-2/vision+voice/voice_of_the_patient.py

```

# Step 1 : setup audio recorder (ffmpeg,portaudio,pyaudio)
import logging
import speech_recognition as sr
from pydub import AudioSegment
from io import BytesIO

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

```

```

def record_audio(file_path, timeout=20, phrase_time_limit=None):
    """
    Simplified function to record audio from the microphone and save it as an MP3 file.

    Args:
    file_path (str): Path to save the recorded audio file.
    timeout (int): Maximum time to wait for a phrase to start (in seconds).
    phrase_time_limit (int): Maximum time for the phrase to be recorded (in seconds).
    """
    recognizer = sr.Recognizer()

    try:
        with sr.Microphone() as source:
            logging.info("Adjusting for ambient noise...")
            recognizer.adjust_for_ambient_noise(source, duration=1)
            logging.info("Start speaking now...")

            # Record the audio
            audio_data = recognizer.listen(source, timeout=timeout, phrase_time_limit=phrase_time_limit)
            logging.info("Recording complete.")

            # Convert the recorded audio to an MP3 file
            wav_data = audio_data.get_wav_data()
            audio_segment = AudioSegment.from_wav(BytesIO(wav_data))
            audio_segment.export(file_path, format="mp3", bitrate="128k")

            logging.info(f"Audio saved to {file_path}")

        except Exception as e:
            logging.error(f"An error occurred: {e}")

audio_filepath="patient_voice_test.mp3"
record_audio(file_path=audio_filepath)

#Step2: Setup Speech to text–STT–model for transcription
import os
from groq import Groq
GROQ_API_KEY="" have removed due to security reasons”

# GROQ_API_KEY=os.environ.get("GROQ_API_KEY")
stt_model="whisper-large-v3"
# client=Groq(api_key=GROQ_API_KEY)

# audio_file=open(audio_filepath, "rb")
# transcription=client.audio.transcriptions.create(
#     model=stt_model,
#     file=audio_file,
#     language="en"

```

```

# )
# print(transcription.text)

def transcribe_with_groq(stt_model, audio_filepath, GROQ_API_KEY):
    client=Groq(api_key=GROQ_API_KEY)

    audio_file=open(audio_filepath, "rb")
    transcription=client.audio.transcriptions.create(
        model=stt_model,
        file=audio_file,
        language="en"
    )

    return transcription.text

```

Phase-2/vision+voice/voice_of_the_doctor.py

```

# Step 1a:Setup text to speech-TTS-model(gTTS )
import os
from gtts import gTTS

# def text_to_speech_with_gtts_old(input_text,output_filepath):
#     language="en"

#     audioobj=gTTS(
#         text=input_text,
#         lang=language,
#         slow=False
#     )
#     audioobj.save(output_filepath)

# input_text="hello all , Myself Pranjali Tiwari"
# text_to_speech_with_gtts(input_text=input_text,output_filepath="ggts_testing.mp3")

# Step 1b : Setup text to speech- TTS model with ElevenLabs

import elevenlabs

from elevenlabs.client import ElevenLabs

ELEVENLABS_API_KEY="have removed due to security reasons"

# def text_to_speech_with_elevenlabs_old(input_text,output_filepath):

#     client=ElevenLabs(api_key=ELEVENLABS_API_KEY)
#     audio=client.generate(
#         text=input_text,

```



```

#     voice="Aria",
#     output_format="mp3_22050_32",
#     model="eleven_turbo_v2"
# )

# elevenlabs.save(audio,output_filepath)

# text_to_speech_with_elevenlabs(input_text,output_filepath="elevenlabs_testing.mp3")

# Step2:Use Model for text output to voice

import subprocess
import platform
from pydub import AudioSegment

#
def text_to_speech_with_gtts(input_text, output_filepath_mp3):
    language = "en"

    # Generate speech and save as MP3
    audioobj = gTTS(
        text=input_text,
        lang=language,
        slow=False
    )
    audioobj.save(output_filepath_mp3)

    # Convert MP3 to WAV (for Windows SoundPlayer)
    output_filepath_wav = output_filepath_mp3.replace(".mp3", ".wav")
    subprocess.run(["ffmpeg", "-i", output_filepath_mp3, output_filepath_wav, "-y"],
        stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

    # Detect OS and play the audio
    os_name = platform.system()
    try:
        if os_name == "Darwin": # macOS
            subprocess.run(['afplay', output_filepath_mp3])
        elif os_name == "Windows": # Windows (Use WAV file for compatibility)
            subprocess.run(['powershell', '-c', f'(New-Object Media.SoundPlayer
"{output_filepath_wav}").PlaySync();'])
        elif os_name == "Linux": # Linux
            subprocess.run(['aplay', output_filepath_mp3]) # Alternative: use 'mpg123' or 'ffplay'
        else:
            raise OSError("Unsupported operating system")
    except Exception as e:
        print(f"An error occurred while trying to play the audio: {e}")

# Example usage
input_text = "Hello all, myself Pranjali Tiwari, new version of testing, autoplay to be precise."

```

```

# text_to_speech_with_ggts(input_text, "ggts_testing_autoplay.mp3")

# using elevenlabs ( testing -1 )

# def text_to_speech_with_elevenlabs(input_text, output_filepath):
#     client=ElevenLabs(api_key=ELEVENLABS_API_KEY)
#     audio=client.generate(
#         text= input_text,
#         voice= "Aria",
#         output_format= "mp3_22050_32",
#         model= "eleven_turbo_v2"
#     )
#     elevenlabs.save(audio, output_filepath)
#     os_name = platform.system()
#     try:
#         if os_name == "Darwin": # macOS
#             subprocess.run(['afplay', output_filepath])
#         elif os_name == "Windows": # Windows
#             subprocess.run(['powershell', '-c', f'(New-Object Media.SoundPlayer
"{output_filepath}").PlaySync();'])
#         elif os_name == "Linux": # Linux
#             subprocess.run(['aplay', output_filepath]) # Alternative: use 'mpg123' or 'ffplay'
#         else:
#             raise OSError("Unsupported operating system")
#     except Exception as e:
#         print(f"An error occurred while trying to play the audio: {e}")

# def text_to_speech_with_elevenlabs(input_text, output_filepath_mp3):
#     client = ElevenLabs(api_key=ELEVENLABS_API_KEY)

#     # Generate speech and save as MP3
#     audio = client.generate(
#         text=input_text,
#         voice="Aria",
#         output_format="mp3_22050_32",
#         model="eleven_turbo_v2"
#     )

#     # Save MP3
#     with open(output_filepath_mp3, "wb") as f:
#         f.write(audio)

#     # Convert MP3 to WAV (Required for Windows)
#     output_filepath_wav = output_filepath_mp3.replace(".mp3", ".wav")
#     subprocess.run(["ffmpeg", "-i", output_filepath_mp3, output_filepath_wav, "-y"],
stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

```

```

# # Example usage
# input_text = "Hello, this is a test of ElevenLabs autoplay feature."
# text_to_speech_with_elevenlabs(input_text, "elevenlabs_testing_autoplay.mp3")
# text_to_speech_with_elevenlabs(input_text, output_filepath="elevenlabs_testing_autoplay.mp3")

# text_to_speech_with_elevenlabs(input_text,output_filepath="1elevenlabs_testing.mp3")

# testing -3

def text_to_speech_with_elevenlabs(input_text, output_filepath_mp3="final.mp3"):
    client = ElevenLabs(api_key=ELEVENLABS_API_KEY)

    # Generate MP3 speech
    audio = client.generate(
        text=input_text,
        voice="Aria",
        output_format="mp3_22050_32",
        model="eleven_turbo_v2"
    )
    elevenlabs.save(audio, output_filepath_mp3)

    # Convert MP3 to WAV (Windows SoundPlayer requires WAV)
    output_filepath_wav = output_filepath_mp3.replace(".mp3", ".wav")
    sound = AudioSegment.from_mp3(output_filepath_mp3)
    sound.export(output_filepath_wav, format="wav")

    # Detect OS and play the correct file format
    os_name = platform.system()
    try:
        if os_name == "Darwin": # macOS
            subprocess.run(['afplay', output_filepath_mp3])
        elif os_name == "Windows": # Use WAV for compatibility
            subprocess.run(['powershell', '-c', f'(New-Object Media.SoundPlayer
"{output_filepath_wav}").PlaySync();'])
        elif os_name == "Linux": # Linux
            subprocess.run(['aplay', output_filepath_mp3]) # Alternative: 'mpg123' or 'ffplay'
        else:
            raise OSError("Unsupported operating system")
    except Exception as e:
        print(f"An error occurred while trying to play the audio: {e}")

    return output_filepath_wav # Return the WAV file for further use

```

Phase-2/vision+voice/db_utils.py

```
import sqlite3
import os

# Consistent DB path
DB_PATH = "reports_1.db"

def init_db():
    print("Initializing database...")
    print("Using database at:", os.path.abspath(DB_PATH))

    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("""
        CREATE TABLE IF NOT EXISTS reports (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            query TEXT NOT NULL,
            response TEXT NOT NULL,
            image_filepath TEXT,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()
    print("Database initialized and table created if not exists.")

def save_to_db(query, response, image_path=None):
    print("Saving to database...")
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute(
        "INSERT INTO reports (query, response, image_filepath) VALUES (?, ?, ?)",
        (query, response, image_path)
    )
    conn.commit()
    conn.close()
    print("Data saved successfully.")
```

Phase-2/vision+voice/UI.py

```
import gradio as gr
from brain import encode_image, analyze_image_with_query
from voice_of_the_patient import record_audio, transcribe_with_groq
from voice_of_the_doctor import text_to_speech_with_gtts, text_to_speech_with_elevenlabs
from db_utils import init_db, save_to_db
import os

GROQ_API_KEY = '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'

# System prompt to guide the doctor model
system_prompt = """You have to act as a professional doctor, i know you are not but this is for learning
purpose.
    What's in this image?. Do you find anything wrong with it medically?
    If you make a differential, suggest some remedies for them. Donot add any numbers or special
characters in
    your response. Your response should be in one long paragraph. Also always answer as if you are
answering to a real person.
    Donot say 'In the image I see' but say 'With what I see, I think you have ....'
    Dont respond as an AI model in markdown, your answer should mimic that of an actual doctor not an
AI bot,
    Keep your answer concise (max 2 sentences). No preamble, start your answer right away please"""

# Initialize the database
init_db()

def process_inputs(audio_filepath, image_filepath):
    print("Processing inputs...")

    # Step 1: Transcribe speech
    speech_to_text_output = transcribe_with_groq(
        GROQ_API_KEY=GROQ_API_KEY,
        audio_filepath=audio_filepath,
        stt_model="whisper-large-v3"
    )

    # Step 2: Analyze image with transcribed text
    if image_filepath:
        encoded_img = encode_image(image_filepath)
        doctor_response = analyze_image_with_query(
            query=system_prompt + speech_to_text_output,
            encoded_image=encoded_img,
            model="meta-llama/llama-4-scout-17b-16e-instruct"
        )
```

```

else:
    doctor_response = "No image provided for analysis."

# Step 3: Convert response to voice using gTTS
voice_of_doctor = text_to_speech_with_gtts(input_text=doctor_response,
output_filepath_mp3="doctor_response.mp3")

# Step 4: Save interaction to database
save_to_db(speech_to_text_output, doctor_response, image_filepath)

return speech_to_text_output, doctor_response, voice_of_doctor

# Create the interface
iface = gr.Interface(
    fn=process_inputs,
    inputs=[
        gr.Audio(sources=["microphone"], type="filepath"),
        gr.Image(type="filepath")
    ],
    outputs=[
        gr.Textbox(label="Speech to Text"),
        gr.Textbox(label="Doctor's Response"),
        gr.Audio("doctor_response.mp3")
    ],
    title="Medical AI Assistant"
)

iface.launch(debug=True)

```

Output

Medical AI Assistant for answering healthcare related queries

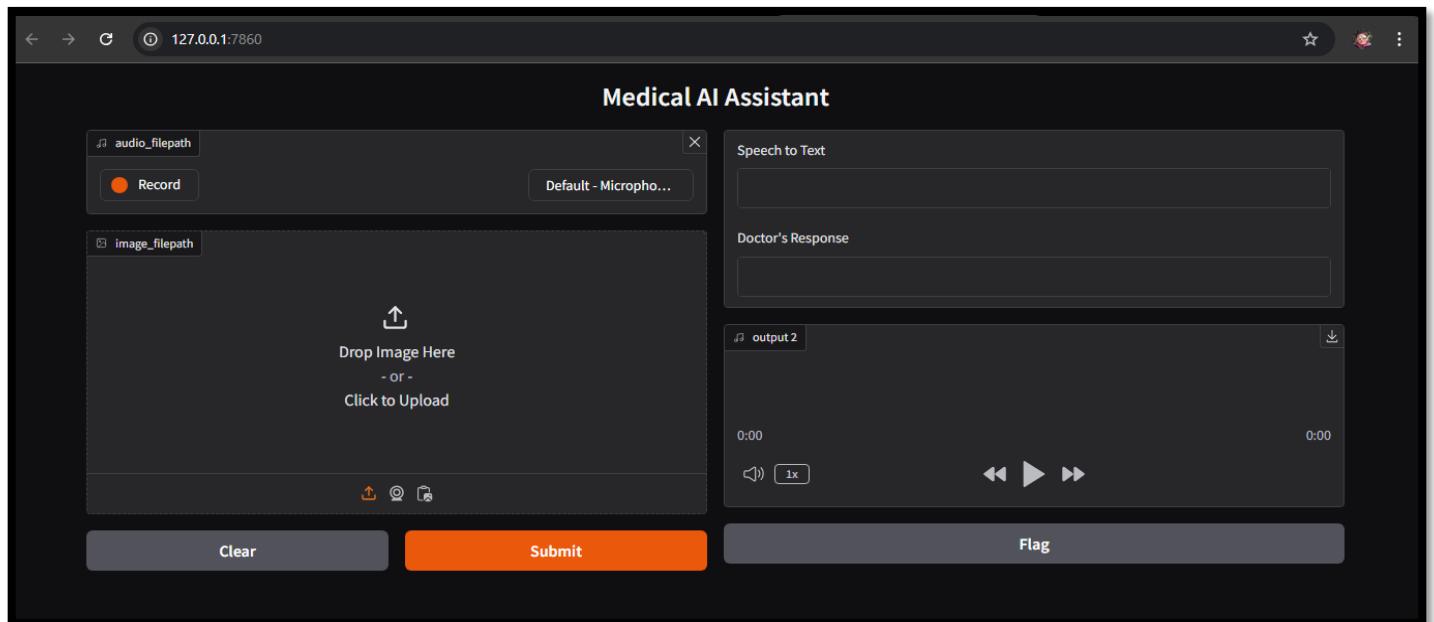


Figure 1: This is the UI interface for Medical AI Assistance ,in which there is a speech recorder, image uploading section, and the output column for speech and text

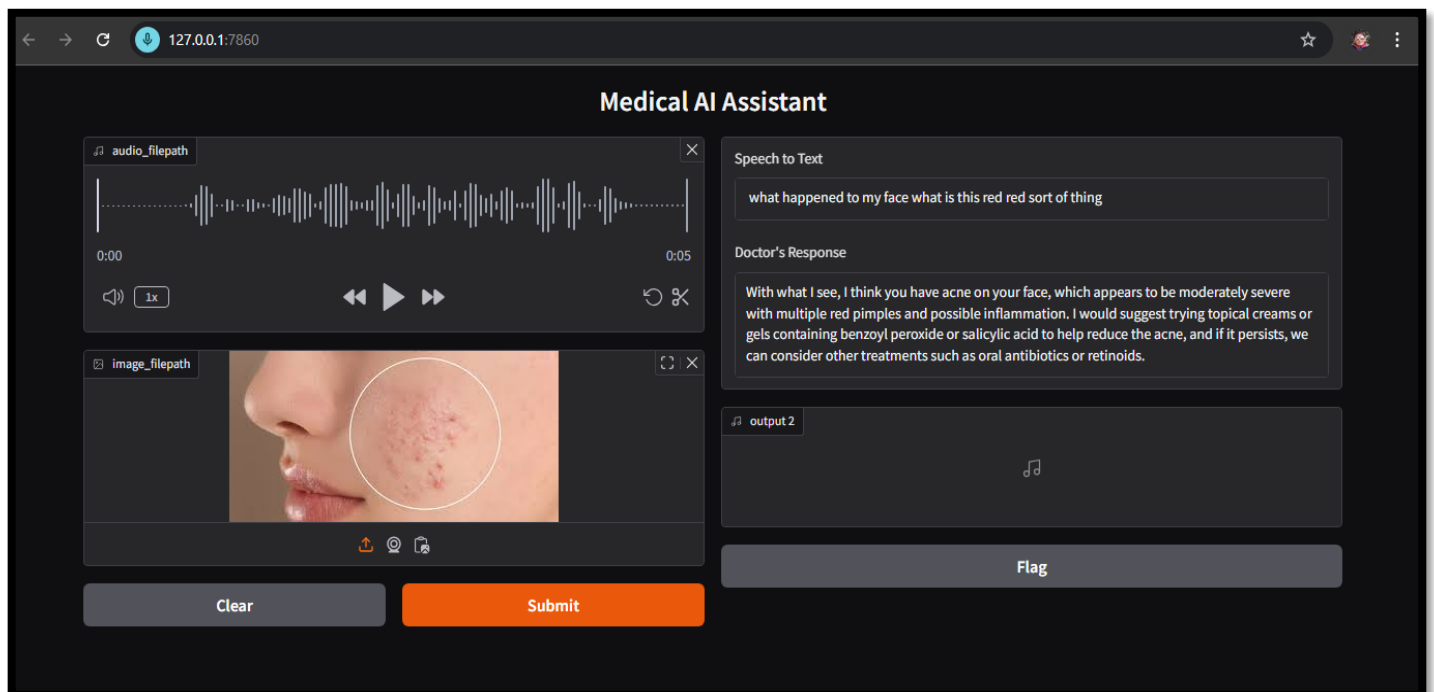


Figure 2: making use of the AI assistant , where the user will input an image and a speech , and the assistant will give teh output in the form of text as well as speech

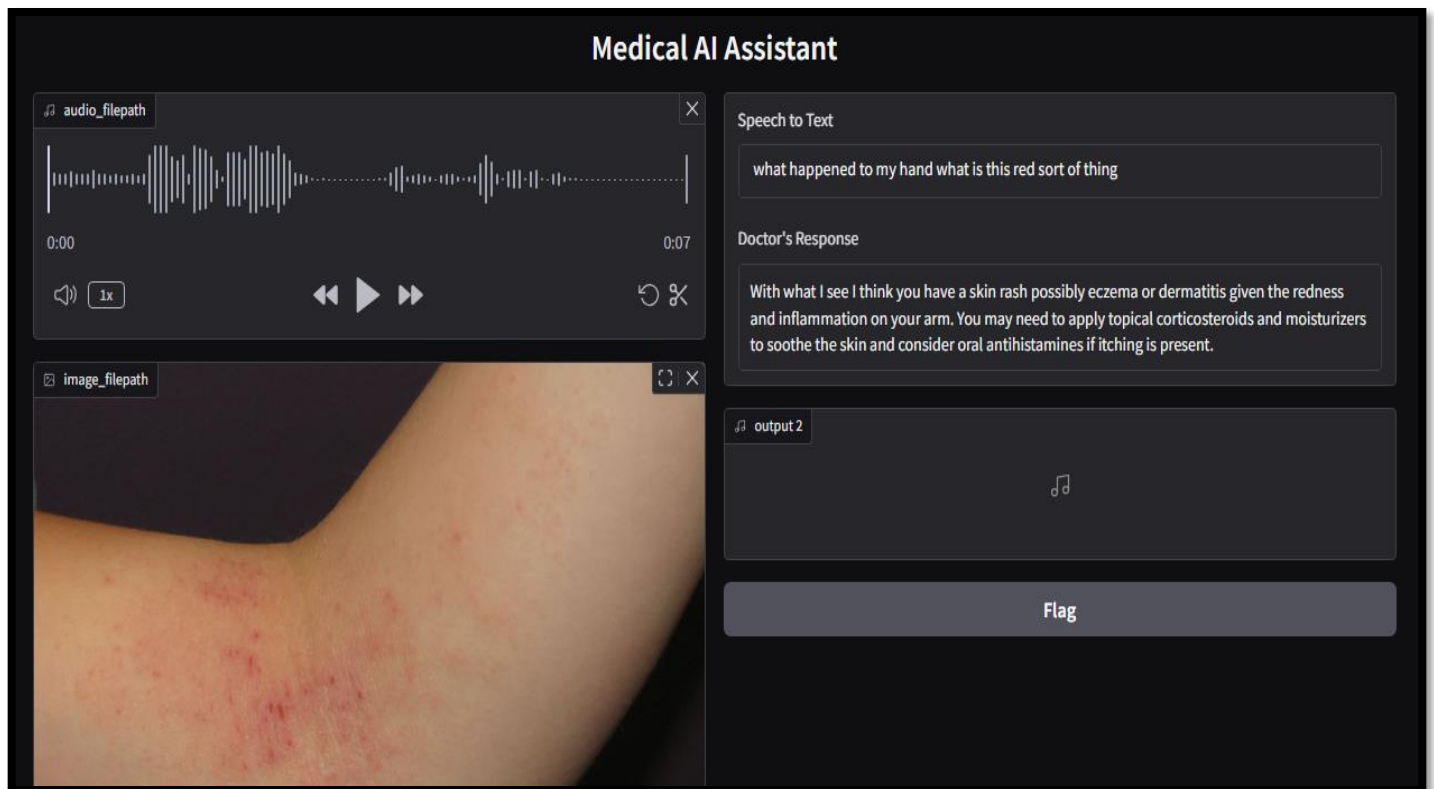


Figure 3: another example of the AI assistant , where the user will input an image and a speech , and the assistant will give teh output in the form of text as well as speech

query	response	image_filepath	timestamp
1 ng with my face? Kindly te...	With what I see, I think you have a skin condition...	C:...	2025-04-30 09:39:24
2 to my hand? What is this ...	With what I see, I think you have a skin condition...	C:...	2025-04-30 09:51:50
3 ng with my hand? Please ...	With what I see, I think you have a skin irritation ...	C:...	2025-05-06 14:38:57
4 face what happened to m...	With what I see, I think you have a severe case ...	C:...	2025-05-06 14:41:18
5 face what is this red red ...	With what I see, I think you have acne on your ...	C:...	2025-05-06 14:45:21

Figure 4: View of the database where the query ,response and the image_path are getting stored

Optical Character Recognition for detecting multiple diseases from the reports

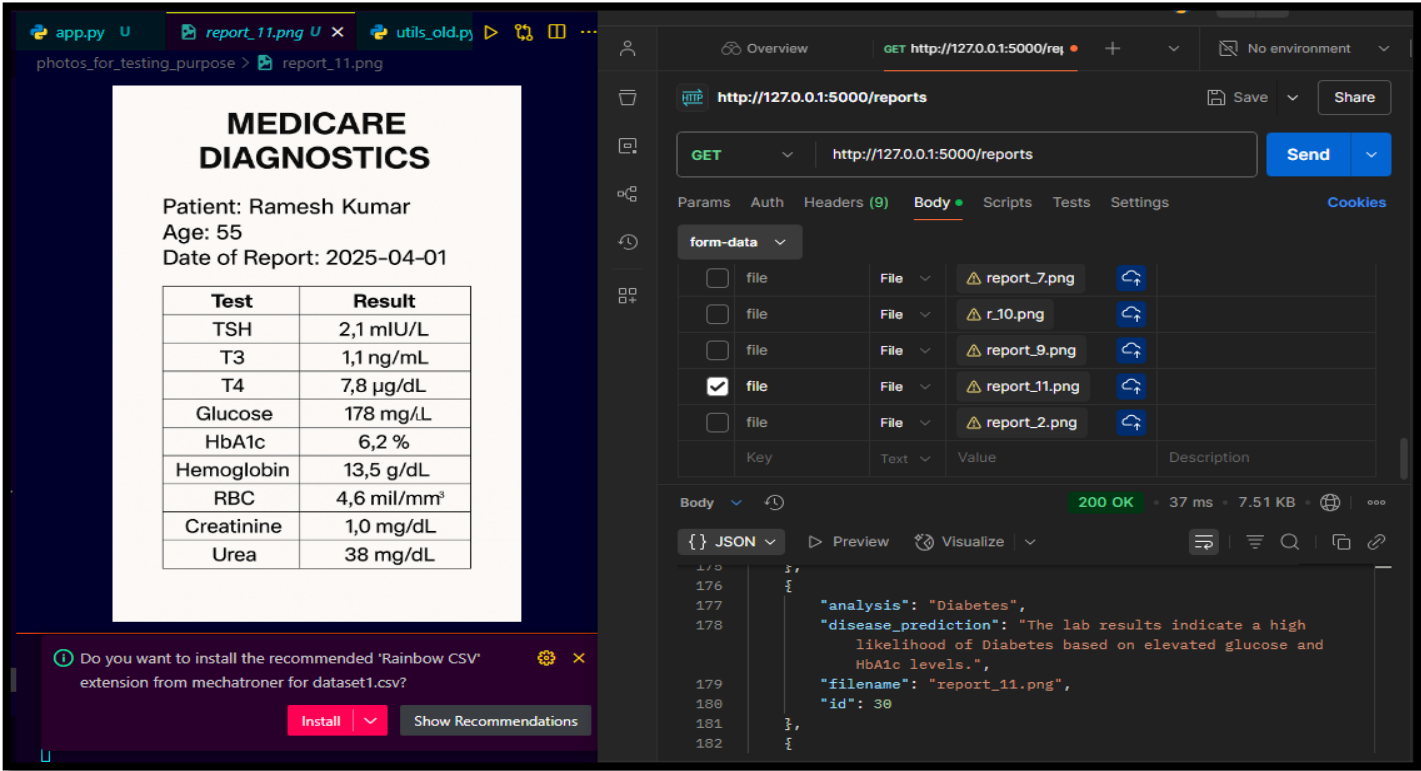


Figure 5: the APIs are being tested on Postman

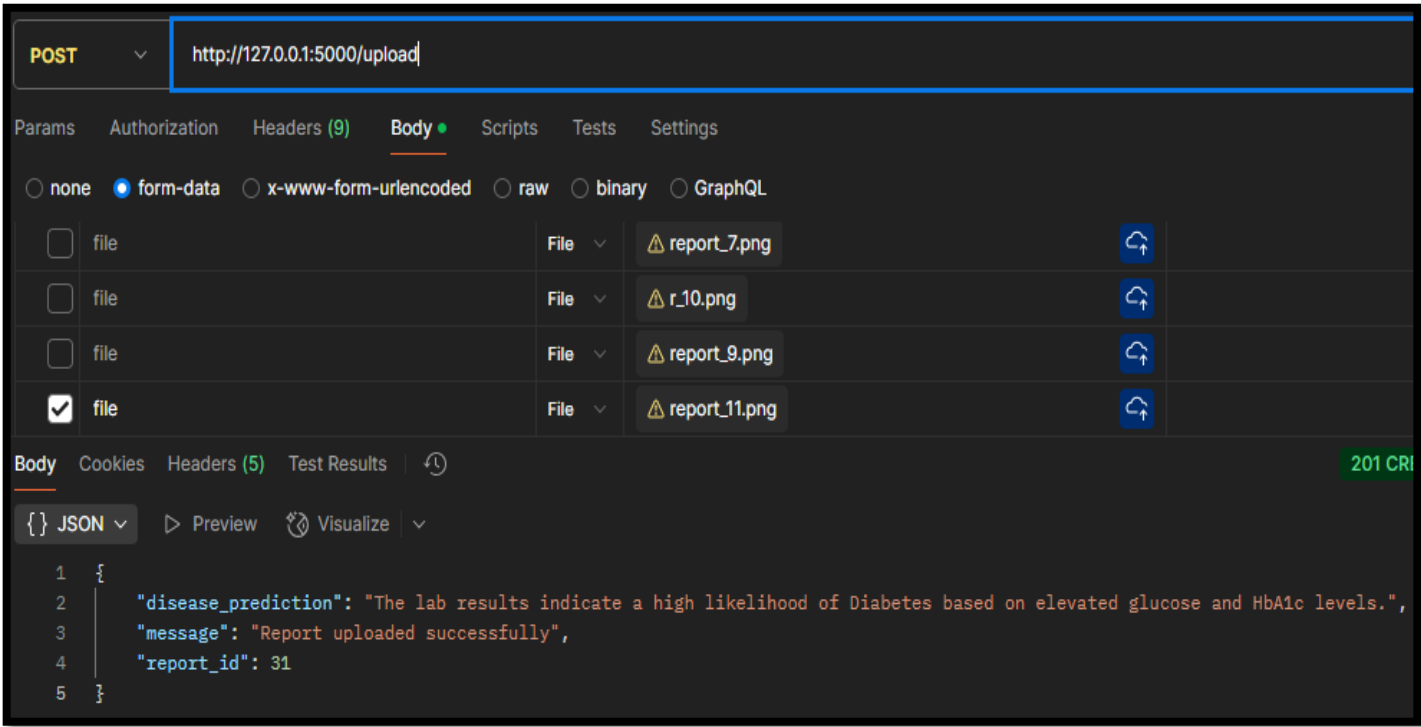


Figure 6: Uploading the Report for testing the OCR system

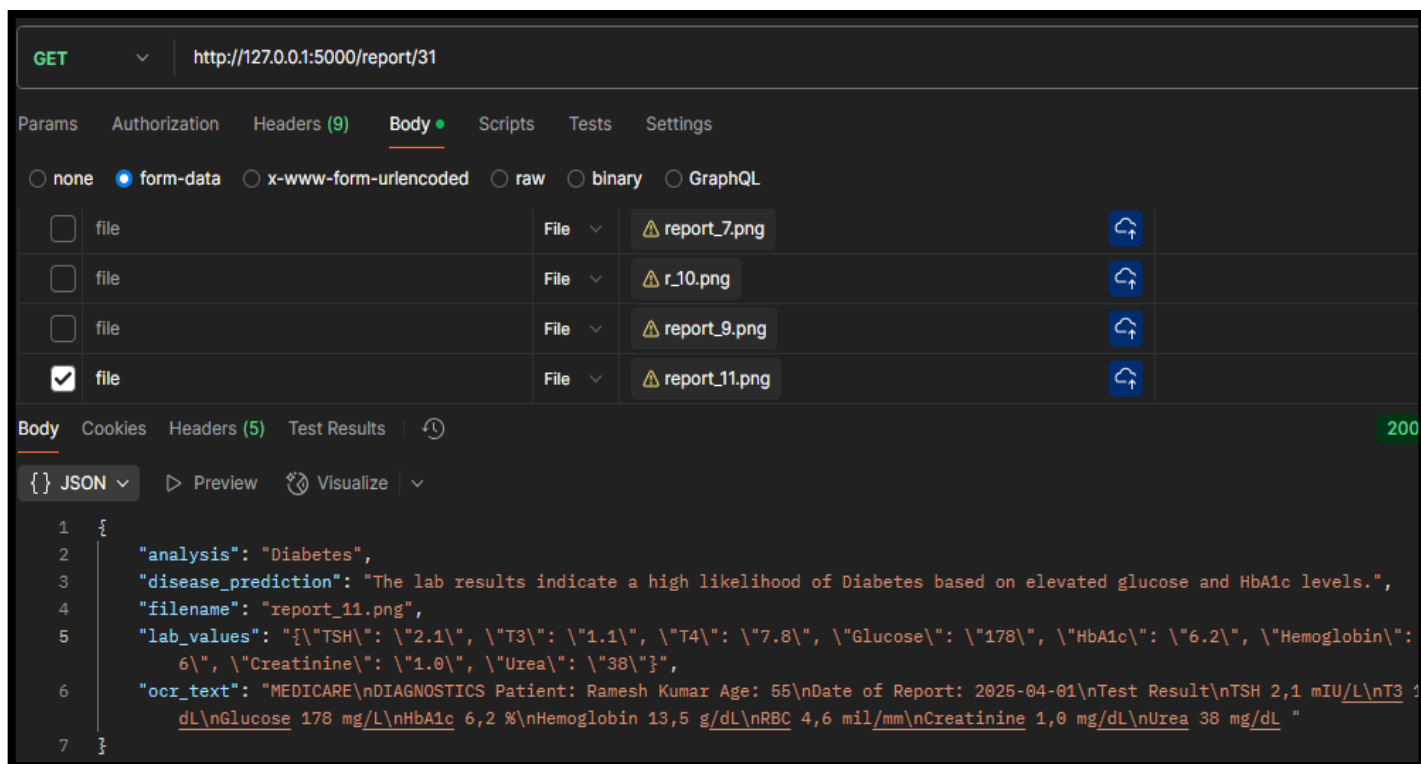


Figure 7: Accessing a particular report through it's unique ID

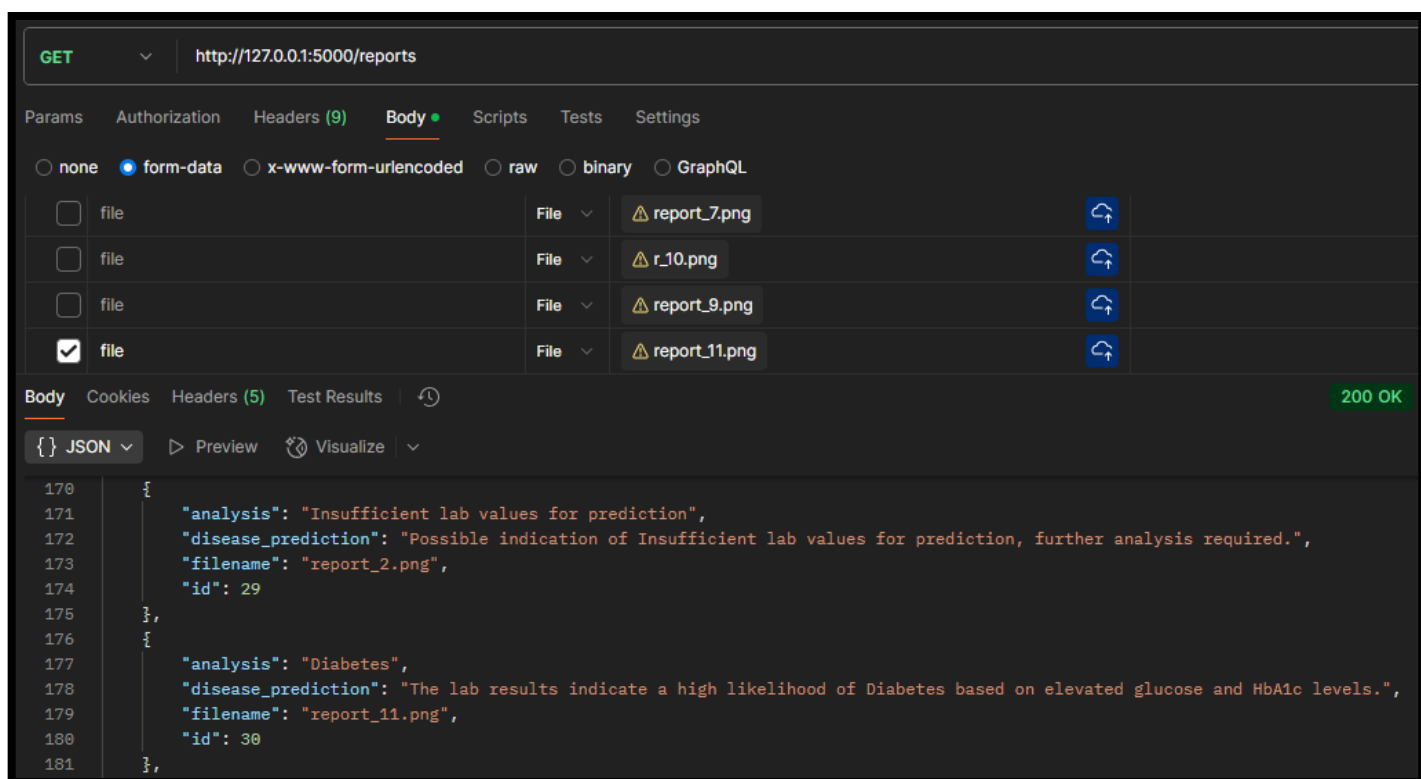


Figure 8: Report slug for accessing all the uploaded reports

1. Conclusion and Future Scope

Conclusion

The development of MediAI marks a significant step toward modernizing diagnostic practices using artificial intelligence. By integrating technologies such as Optical Character Recognition (OCR), Machine Learning, Deep Learning, and Retrieval-Augmented Generation (RAG), the system demonstrates the capability to accurately extract, interpret, and analyze medical data. The implementation of vision- and voice-based multimodal interfaces further enhances user interaction, making MediAI accessible and intuitive for both patients and healthcare professionals. The system successfully streamlines early detection for diseases like diabetes and thyroid disorders, supporting clinicians with timely insights and reducing human error in diagnostics. Overall, MediAI holds the potential to improve diagnostic accuracy, operational efficiency, and healthcare accessibility—especially in underserved regions.

Future Scope

- **Expansion to More Diseases:** Future iterations of MediAI can include detection for additional medical conditions such as anemia, kidney disease, and liver disorders by incorporating larger and more diverse datasets.
- **Real-Time Monitoring and IoT Integration:** Integrating wearable health devices for real-time data input can enable continuous patient monitoring and proactive healthcare.
- **Integration with EHR Systems:** Linking MediAI with electronic health records (EHR) would provide seamless access to historical data, enhancing the depth of diagnostic insights.
- **Multilingual and Regional Adaptation:** Adding support for regional languages and localized medical terminology can broaden usability in diverse demographic regions.
- **Mobile Application Development:** A mobile-friendly version of MediAI can offer on-the-go diagnostics, increasing accessibility and usability for rural and remote populations.
- **AI-Driven Treatment Recommendations:** Beyond diagnosis, future updates may offer AI-generated treatment suggestions aligned with current medical guidelines and patient history.
- **Voice Command and NLP Refinement:** Further refinement in speech recognition and natural language processing will improve the accuracy of voice-based inputs and chatbot responses.

2. References

- [1] <https://huggingface.co/docs/transformers/index>
- [2] <https://flask.palletsprojects.com/en/latest/>
- [3] <https://scikit-learn.org/stable/>
- [4] <https://github.com/tesseract-ocr/tesseract>
- [5] <https://pypi.org/project/pytesseract/>
- [6] <https://www.geeksforgeeks.org/python-build-a-ocr-based-medical-report-analyzer-using-tesseract/>
- [7] <https://www.kaggle.com/datasets>
- [8] <https://www.analyticsvidhya.com/blog/2021/07/build-your-own-ocr-application-using-tesseract-and-python/>
- [9] <https://realpython.com/flask-by-example-part-1-project-setup/>
- [10] <https://www.section.io/engineering-education/building-a-restful-api-using-flask/>
- [11] <https://docs.opencv.org/>
- [12] <https://medium.com/mlearning-ai/ocr-using-tesseract-and-python-3d8c5ea6b5c1>
- [13] <https://towardsdatascience.com/creating-a-medical-diagnosis-predictor-using-machine-learning-e198f24c9389>
- [14] <https://www.gradio.app/>
- [15] <https://huggingface.co/docs/transformers/llms>
- [16] <https://huggingface.co/blog/rag>